

Repif

DOCUMENTATION

OLEKSANDR CHEREPKOV

Table of content

A1 PVC HOLDER	5
Assignment	5
Part 1: Dimensional Layout and Preparation	6
Part 2 Execution of Drilling	7
A2 PCB.....	8
Assignment	8
Work Steps	9
2.1 Component Selection	9
2.2 Assembly and Soldering.....	10
Material List	11
A3 POWER SUPPLY	12
Task Description.....	12
Procedure.....	12
Cable Preparation	12
Staggered Cutting.....	13
Soldering the Conductors and Shielding.....	13
Result.....	14
A4 SENSOR 1.....	15
Task Description.....	15
Sensor installing	17
Sensor Testing	17
Materials Used.....	19
A5 SENSOR 2.....	20
Task Description.....	20
Sensor installing	21
Sensor Testing	22
Materials Used.....	25
A6 ASSEMBLY.....	25
Task Description.....	25
Part 1: Housing Assembly.....	25
Part 2: Electronics Installation	25

A7 LIGHTING	28
Task description	28
LED strip preparation.....	28
Housing preparation	29
Connecting to PCB.....	29
LED strip mounting.....	30
L1 Raspberry Pi.....	31
Install and Configure	31
Install "Raspberry Pi OS (64-bit)" on the microSD card.....	31
Configure VNC	32
Configure static IP for Ethernet	34
Enabling I ² C	34
Install and configure screen.....	35
Docker	36
Docker installation	36
Firmware and website files	36
Config.toml file configuration.....	38
Creating Docker container	38
Configure autostart.....	39
Resources.....	40
L2 DATABASE	40
Conceptual Data Model (CDM).....	40
Logical Data Model (LDM)	41
Creating Database.....	42
L3 WEBSITE	43
Introduction	43
Authentication and Security	43
Login and Authentication.....	43
Registration and Email Verification.....	44
User Workspace	44
Personal Dashboard.....	44
Profile Management and Customization.....	44

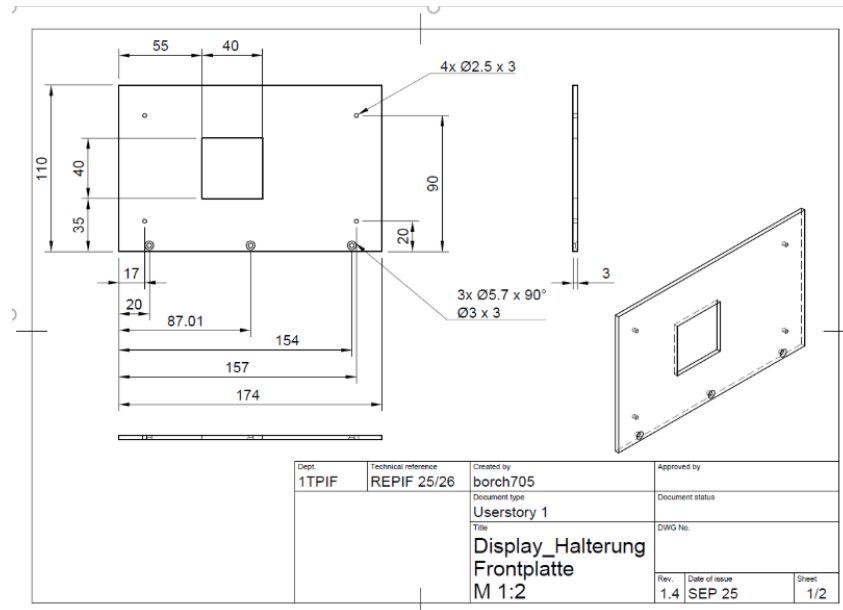
Public Profile View	44
IoT and Data Management	44
Station Management	44
Measurement Visualization and Analytics	44
Collaborative Features	45
Collections and Data Sharing	45
Social Integration (Friends)	45
Real-Time Communication (Chat)	45
System Administration and UI	45
Administrator Control Panel	45
Notification System (Bell Icon)	45
Localization and Responsive Design	45
L4 WEB-DATABASE SERVER	46
Introduction	46
System Architecture	46
Environment and Docker Installation	47
Virtual Machine Preparation	47
SSH Installation and Configuration	47
Docker Engine Installation	48
Docker Compose Installation	48
Directory Structure Preparation	48
Project Structure	49
Description of Each Directory	49
Docker Compose Stack	50
docker-compose.yml	50
Explanation of the Stack	52
Using the Stack	52
Environment Variables	53
.env File Contents	53
Apache Configuration and VirtualHost	53
VirtualHost Configuration File	53
Enabling Required Apache Modules	54

Activating the VirtualHost	54
Database Initialization and Security.....	55
Database Initialization Script	55
Running mysql_secure_installation	55
PHP Example Application	56
index.php	56
dbtest.php	56
Mailhog Configuration and Email Testing	57
Installing Composer	57
PHPMailer Test Script	58
Resources.....	60
L5 DATA TRANSFER.....	60
Introduction	60
Task Description.....	61
Database Preparation.....	61
PHP Data Transfer Script.....	63
Test Submission Script.....	69
Manual Input Form.....	70
L6 BACKUP SERVER.....	71
Web-Database Server Configuration	71
1.1. Docker & MariaDB Networking.....	72
1.2. Database User Privileges.....	72
1.3. Sudoers Configuration for Full OS Backup.....	72
Backup Server Configuration.....	72
2.1. Service Installation	72
2.2. Secure SSH Key Authentication	73
2.3. FTP Isolation (Chroot Jail).....	73
Automation & Scripting.....	73
3.1. Database & Files Backup (backup.sh)	73
3.2. Full OS Backup (os_backup.sh).....	73
3.3. Crontab Schedule	73
Disaster Recovery.....	73

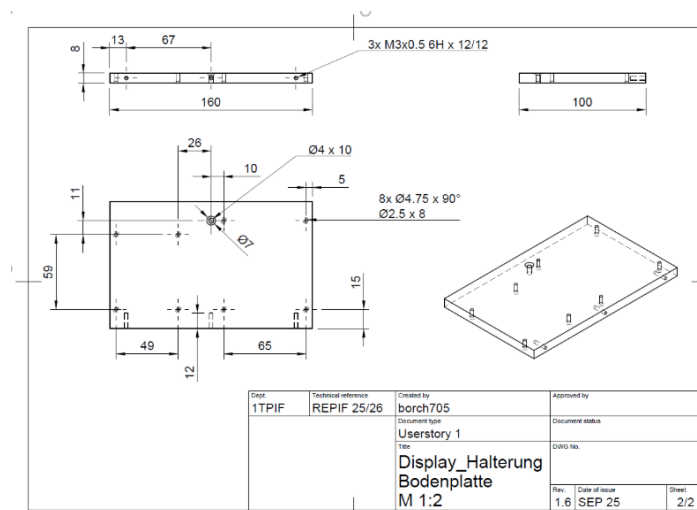
A1 PVC HOLDER

Assignment

This assignment required the fabrication of two PVC components: a base plate and a display bracket. The task involved drilling holes of specified diameters, preparing openings for cable routing, deburring edges, and assembling the components with the touch display and Raspberry Pi. All operations had to be performed in accordance with workshop safety regulations and professional standards.



Picture 1.1. Technical Drawing of Display Holder Front Plate



Picture 1.2. Technical Drawing of Display Holder Base Plate

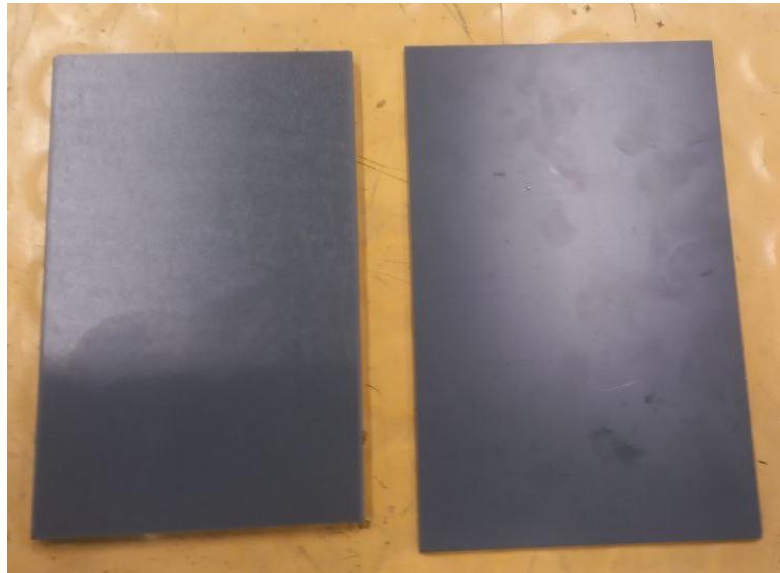
Part 1: Dimensional Layout and Preparation

The first stage consisted of marking and transferring the specified measurements onto the PVC base plate and the display bracket. The process was carried out with precision to ensure that the subsequent drilling operations would align correctly with the Raspberry Pi, the circuit board, and the display holder.

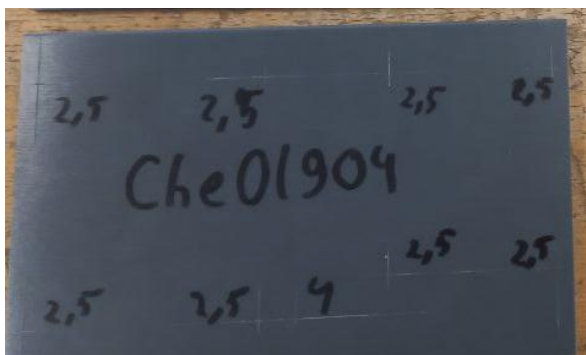
The marking procedure involved several steps:

- Reference points for the future holes were inscribed directly onto the PVC surface using a scribe.
- Diameters of the holes were annotated on the material to avoid confusion during drilling.
- Each manufactured part was labeled with the personal IAM code to ensure traceability and compliance with workshop requirements.

This careful preparation ensured that the drilling stage could be executed with accuracy and minimized the risk of misalignment.



Picture 1.3. Initial PVC plates



Picture 1.4. Markered PVC plates

Part 2 Execution of Drilling

The second stage of the assignment involved drilling holes of specified diameters into the PVC base plate and the display bracket. This operation was intended to prepare the components for assembly with the Raspberry Pi, the circuit board, and the touch display.

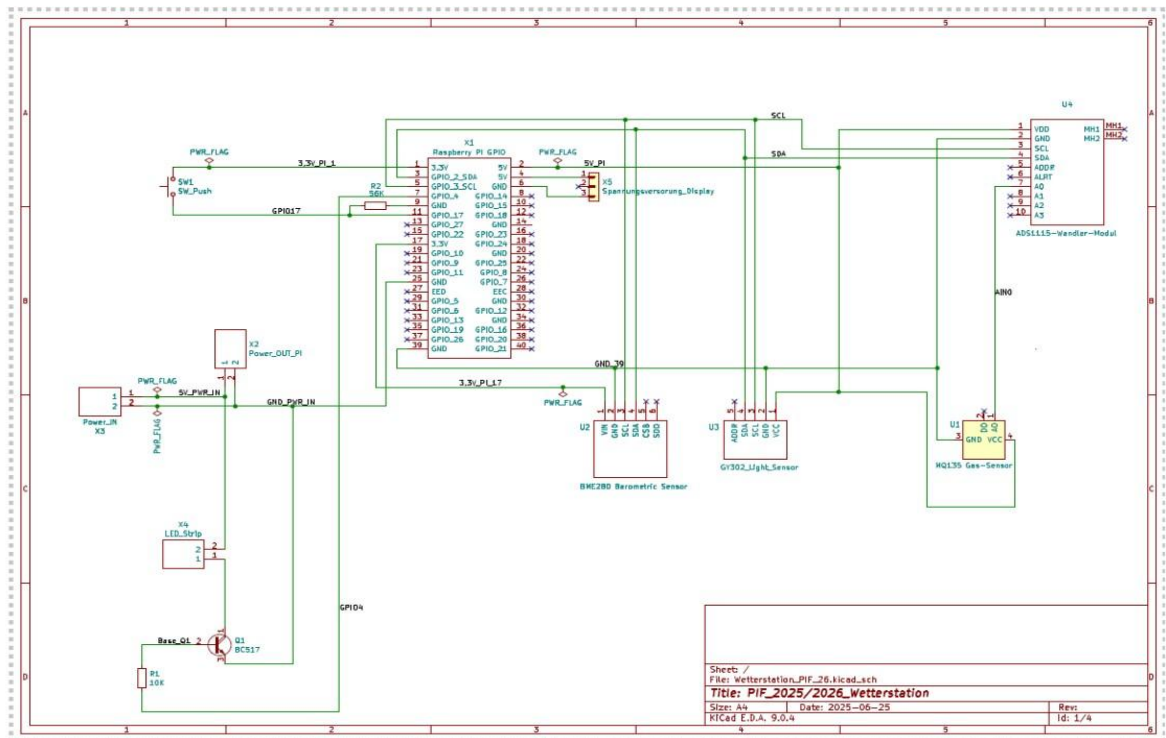
The following steps were successfully completed:

- Holes of 2.5 mm diameter were drilled in the PVC base plate for attaching the Raspberry Pi and the circuit board.
- A countersink hole of 4 mm diameter was drilled to accommodate a wood screw for later attachment of the base plate to the wooden housing.
- Holes of 2.5 mm diameter were drilled for attaching the display.
- Holes of 3 mm diameter were drilled in the PVC display bracket for the M3 countersunk screws connecting it to the base plate.

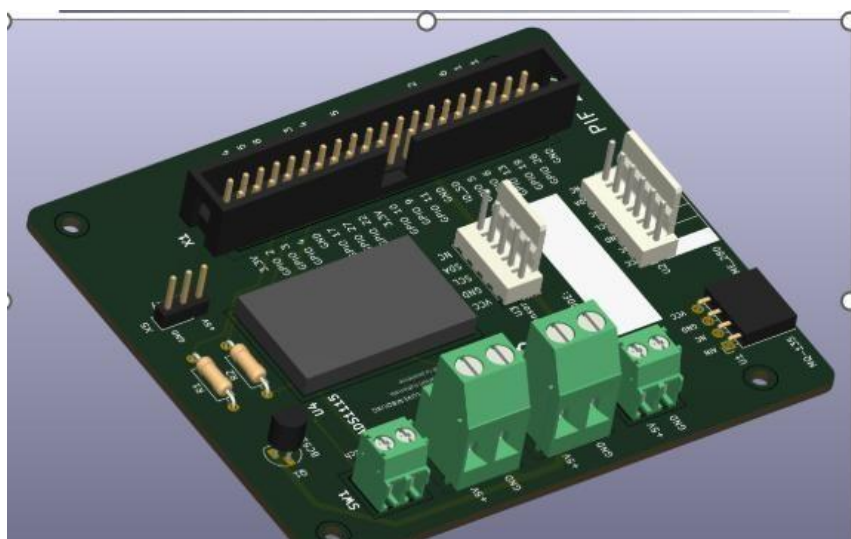
A2 PCB

Assignment

The goal of this assignment was to manufacture a printed circuit board (PCB) that serves as an interface between the Raspberry Pi, sensors, and actuators. According to the work order, the PCB must include all required components (except sensors), which must be professionally mounted and soldered. The board must be screwed onto the PVC base plate using 5 mm spacers, and all safety regulations must be followed during the assembly. The PCB must also be marked with the IAM code.



Picture 2.1. Electrical schema of PCB

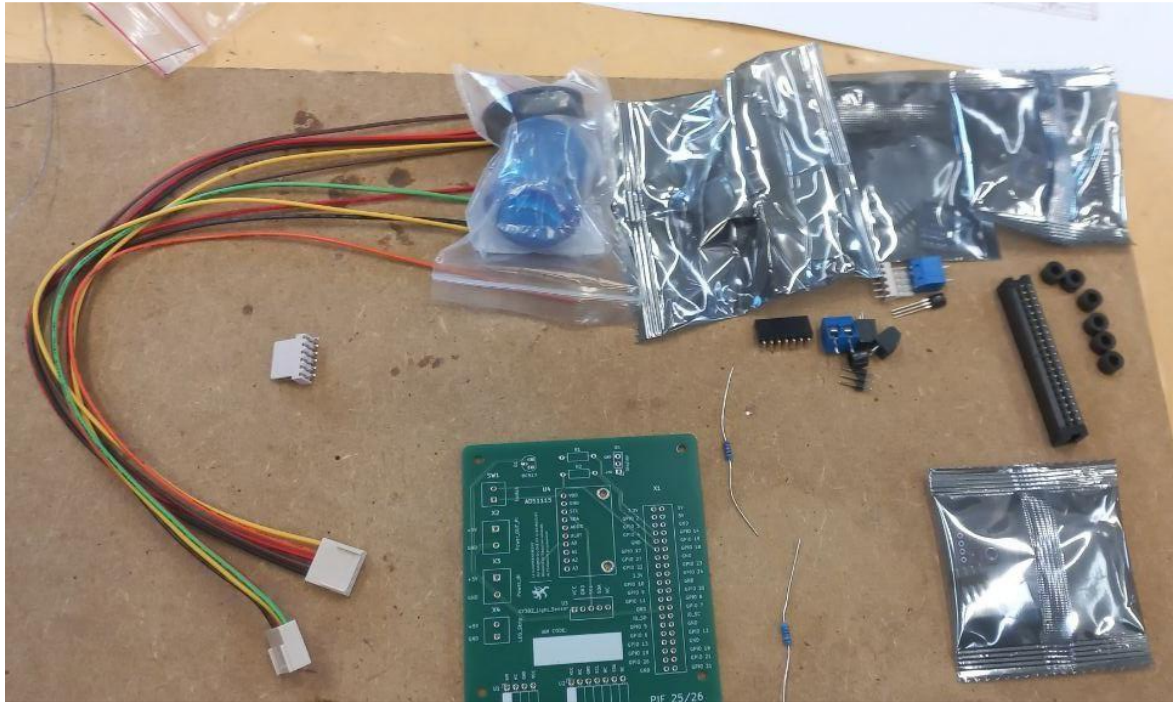


Picture 2.2. PCB model

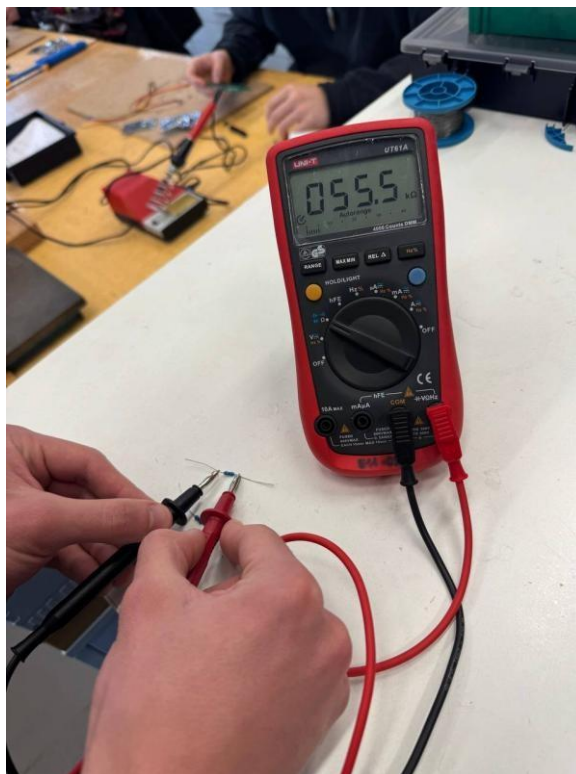
Work Steps

2.1 Component Selection

- All components were selected according to the provided circuit diagram.
- The PCB layout was reviewed to determine correct placement.
- Resistors were verified using multimeter, to ensure correct values.



Picture 2.3. Preselected components



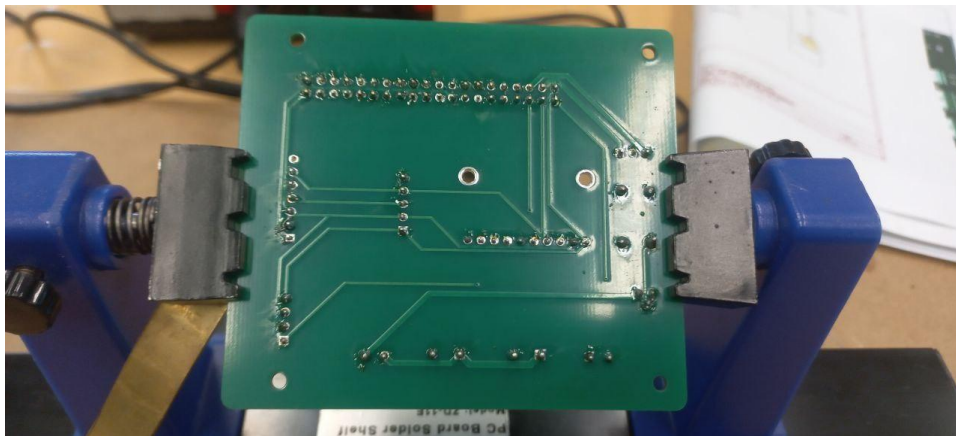
Picture 2.4. Resistors verification

2.2 Assembly and Soldering

- Components were inserted into their designated positions on the PCB.
- Each component was soldered using proper soldering techniques to ensure clean and reliable joints.
- Excess leads were trimmed to avoid short circuits.
- The board was inspected for solder bridges and cold joints.



Picture 2.5. PCB after soldering front



Picture 2.6. PCB after soldering back

Material List

№	Component	Quantity
1	2-layer printed circuit board 75 x 85 mm	1
2	40-pin straight connector strip R WS40G	1
3	GPIO ribbon cable for Raspberry Pi Model B+ (40 pins)	1
4	Metal resistor 0207 0.6W 10K	1
5	Metal resistor 0207 0.6W 56K	1
6	BC517 transistor	1
7	PCB connection terminal 2-pin RM 5	2
8	PCB terminal block, 2-pin RM 3.5	1
9	1-row angled socket strip 1 x 4 - RM2.54mm	1
10	1-row angled female connector strip 1 x 7 - RM2.54mm	1
11	Pin strip, 1 row, straight, 1 x 3 - RM2.54mm	1
12	PCB connector, straight pin header with female connector (set)	1
13	M2.5 x 10 cylinder head	4
14	M2.5 washers	8
15	M2.5 nuts	8
16	5 mm plastic spacer roller	1
17	Gas sensor module MQ-135	1
18	ADS1115 ADC module I2C	1
19	GY-BME280 Barometric Sensor Module	1
20	GY302 / BH1750 Light Sensor Module	1
21	Switching cable 0.5 mm ² black	1
22	Switching cable 0.5 mm ² red	1

A3 POWER SUPPLY

Task Description

The task consisted of repairing a multi-core shielded cable by cutting it in half and reconnecting all internal conductors. The cable contained four individual wires and an external shielding braid. To ensure mechanical reliability and avoid creating a thick solder joint area, the conductors had to be soldered using a staggered splicing technique. After soldering, each connection had to be insulated using heat-shrink tubing, and the shielding braid also needed to be reconnected and covered.



Picture 3.1. Initial Wire.

Procedure

Cable Preparation

The cable was cut, and approximately 3–4 cm of the outer insulation was removed on both sides. This exposed the shielding braid, which was carefully pulled back to provide access to the internal conductors.

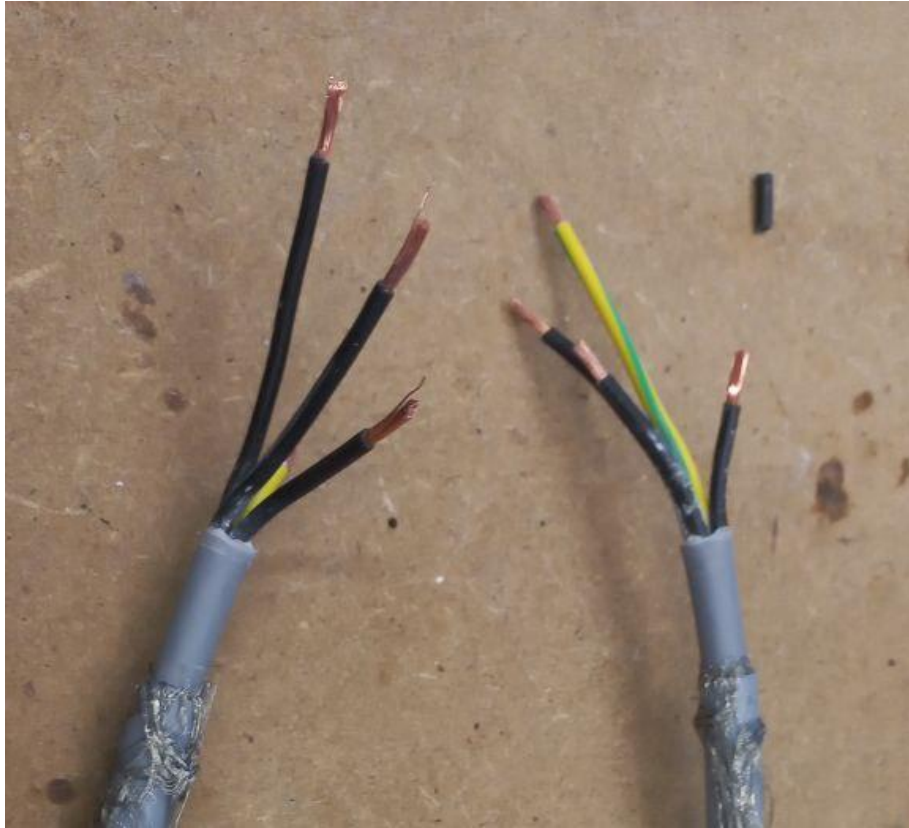
Inside the cable, four wires were found. Each conductor was stripped to expose the copper core.

Staggered Cutting

To prevent all solder points from lining up in the same location, each of the four wires was trimmed with a 1 cm offset relative to the others.

This method ensures:

- a thinner and more flexible repair area,
- reduced risk of cable breakage at the joint,
- better distribution of mechanical stress.



Picture 3.2. Striped and staggered conductors

Soldering the Conductors and Shielding

Each pair of matching wires was twisted together and soldered. The solder joints were smooth and fully covered the copper to ensure low-resistance electrical contact.

After soldering, each connection was immediately insulated using small-diameter heat-shrink tubing.

The shielding braid on both sides was gathered, twisted, and soldered together as well. This restores electromagnetic shielding and prevents electrical noise from entering the signal lines. A larger heat-shrink tube was then placed over the shield splice.

Result

The cable was fully repaired using proper soldering and insulation techniques.

The staggered splicing method ensured a slim, flexible joint without unnecessary thickening.

The shielding was restored, and the entire assembly was strengthened with heat-shrink tubing.

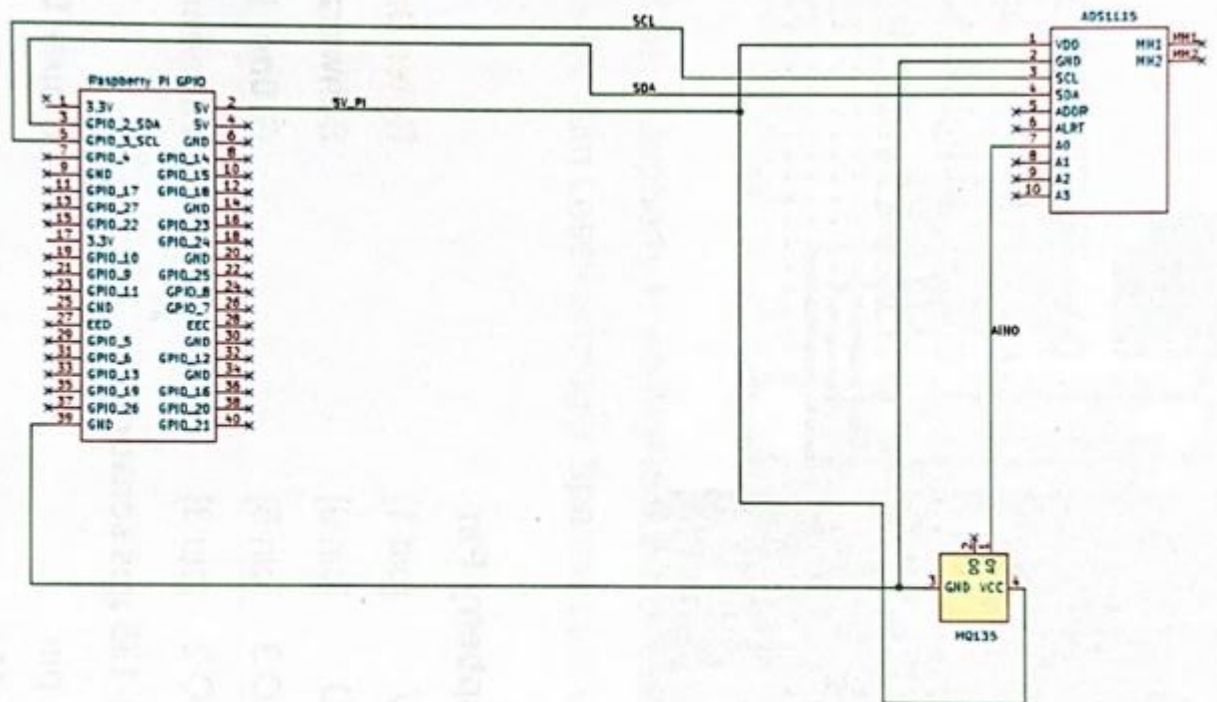
The cable is now functional, mechanically stable, and electrically safe.



Picture 3.3. Soldered and tubed wire.

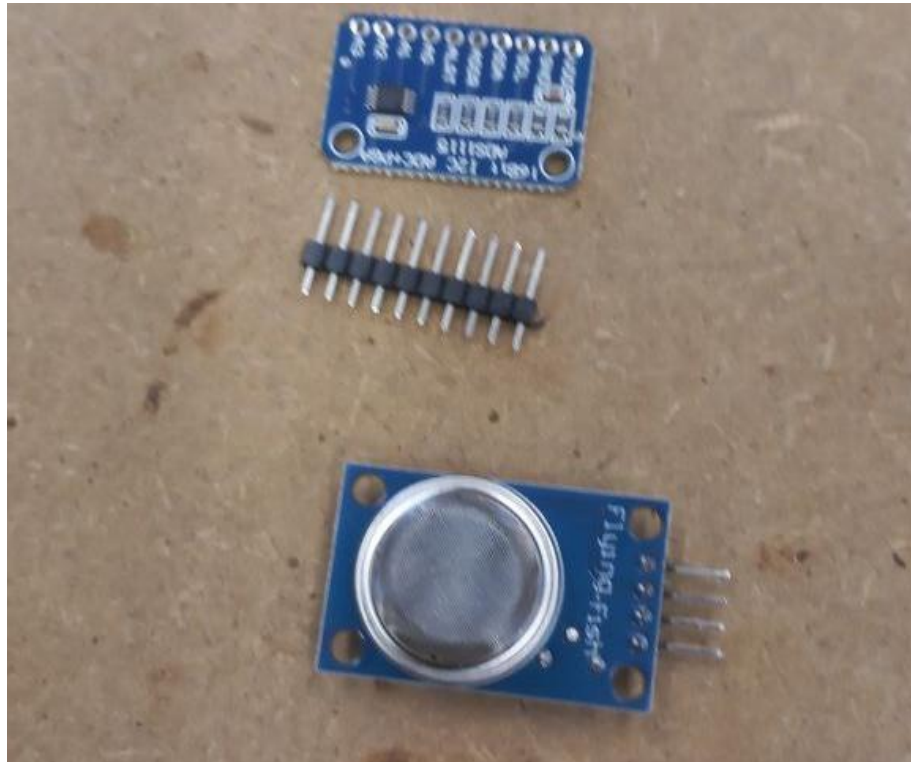
Modul Pin	> Raspberry Pin	
VDD	> 3.3V [pin 1]	Roter Draht
GND	> GND [pin 9]	Schwarzer Draht
SCL	> GPIO 3 [pin 5]	Grüner Draht
SDA	> GPIO 2 [pin 3]	Violetter Draht
Modul Pin	> MQ-135 gas sensor	
A0	> Left pin	Blauer Draht
RaspPi Pin	> MQ-135 gas sensor	
3.3V [pin 17]	> Third pin	Red wire
GND [pin 20]	> Right pin	Schwarzer Draht

Pictures 4.1a and 4.1b. Connecting schema.



Picture 4.2. Electrical schema.

Sensor installing



Picture 4.3. Provided components.

Pin headers were soldered onto the ADS1115 converter module.

The MQ135 gas sensor was inserted into its designated slot on the PCB. Both modules were mounted directly onto the PCB, which is connected to the Raspberry Pi via GPIO.

All connections followed the official schematic provided in the work order (Pictures 1a and 1b). The PCB handles routing between the Raspberry Pi and the sensor modules, including I²C lines and analog input.

Sensor Testing

The I²C address was identified using:

```
sudo i2cdetect -y 1
```

```
cheol904@cheol904:/opt/station $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- 48 -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

Picture 4.4. Result of i2c detecting.

The detected address was then updated in the configuration file of the script.

The Python script was executed to read analog values from the MQ135 sensor via the ADS1115. The system returned valid air-quality readings, confirming:

- Correct module placement
- Proper PCB routing
- Successful I²C communication
- Functional sensor output

```
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2587 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2582 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2579 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2576 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2575 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2554 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2549 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2544 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2542 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2540 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2885 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2906 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2963 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2967 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2954 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2969 ppm
cheol904@cheol904:/opt/station $ python3 test-mq135.py
C02 concentration: 2962 ppm
cheol904@cheol904:/opt/station $
```

Picture 4.5. Python script execution.

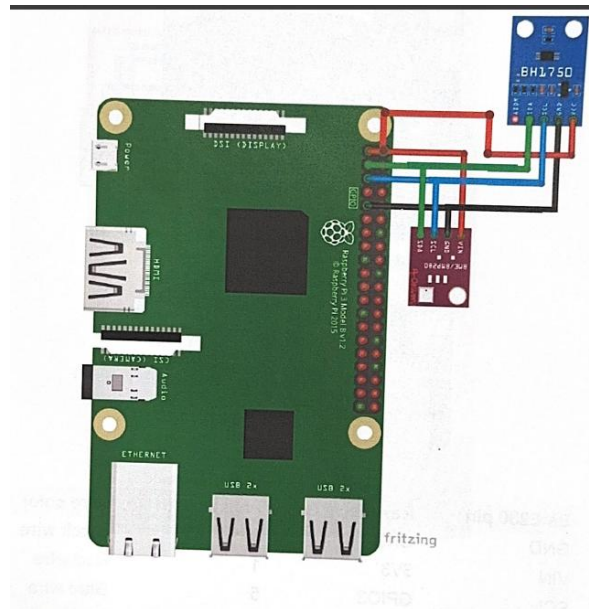
Materials Used

- MQ135 gas sensor
- ADS1115 ADC module
- Raspberry Pi
- Custom PCB (PIF 25/26)
- Pin headers
- Soldering iron and solder

A5 SENSOR 2

Task Description

To connect and test the BH1750 light sensor and the BMP280 pressure/temperature sensor using a custom PCB interfaced with a Raspberry Pi, in order to obtain data on brightness, temperature, and atmospheric pressure.

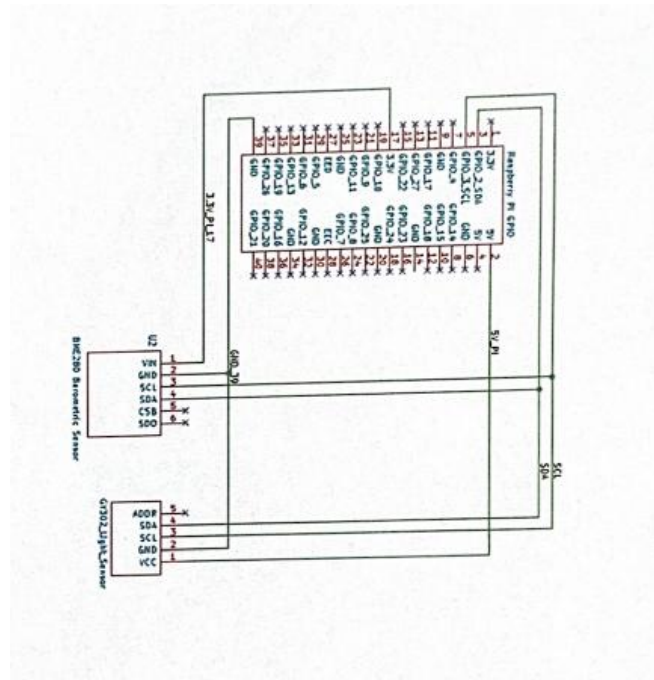


Picture 5.1. Connecting schema.

Modul-Stift	Raspberry Pi Stift	Physikalisch e Nadel	Farbe des Kabels
VCC	5V	2	Rotes Kabel
SDA	GPIO2	3	Grüner Draht
SCL	GPIO3	5	Blaues Kabel
GND	GND	14	Schwarzes

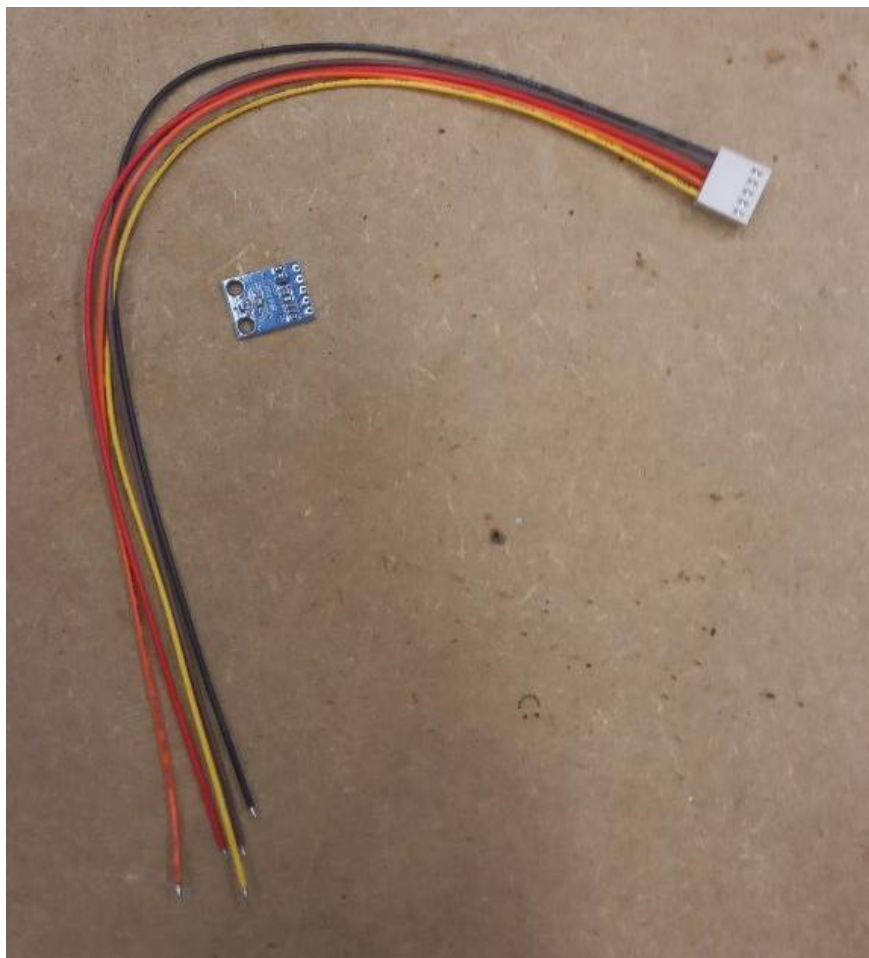
BME280 pin	Raspberry Pi pin	Physical pin No.	Wire color
GND	GND	9	Black wire
VIN	3V3	1	Red wire
SCL	GPIO3	5	Blue wire
SDA	GPIO2	3	Green wire

Pictures 5.2a and 5.2b. Connecting tables.



Picture 5.3. Electrical schema.

Sensor installing



Pictures 5.4a and 5.4b. Provided components.

Socket strips with cables were soldered directly into the pin holes on both sensor modules — BH1750 and BMP280.

The other ends of these cables (white connectors) were inserted into pre-soldered sockets on the custom PCB. The PCB itself is connected to the Raspberry Pi via GPIO.

All connections followed the official schematic provided in the work order. The PCB handles routing between the Raspberry Pi and the sensor modules, including I2C lines and power.

Sensor Testing

The I²C address was identified using:

```
sudo i2cdetect -y 1
```

The detected address was then updated in the configuration file of the script.

Initially, the test script for BMP280 was written for a BME280 sensor. Due to the sensor being replaced with a BMP280, the original script (`test-bme280.py`) was incompatible. The script was modified to work with the BMP280 sensor's register map and communication protocol.

To do that, a virtual environment was created and activated to isolate the Python dependencies:

```
cd /opt/station
sudo apt install -y python3-venv python3-full
python3 -m venv venv
source venv/bin/activate
```

Then, the required library for the BMP280 sensor was installed using pip:

```
pip install bmp280
```



```

cheol904@cheol904:/opt/station $ cat test-bh1750.py
ADDRESS = 0x23 # TODO: Change this

from bh1750 import BH1750
import smbus

def main():
    bus = smbus.SMBus(1) # Use 1 for Raspberry Pi, 0 for older models
    sensor = BH1750(bus, ADDRESS)

    try:
        light_level = sensor.read_light()
        print(f"Light Level: {light_level} lux")
    except Exception as e:
        print(f"Error reading light level: {e}")

if __name__ == "__main__":
    main()
cheol904@cheol904:/opt/station $

```

Picture 5.5. Bh1750 sensor testing script.

```

cheol904@cheol904:/opt/station $ cat test-bmp280.py
from bmp280 import BMP280
from smbus2 import SMBus
import time

ADDRESS = 0x76

def main():
    bus = SMBus(1)
    sensor = BMP280(i2c_dev=bus, i2c_addr=ADDRESS)

    # while True:

        temperature = sensor.get_temperature()
        pressure = sensor.get_pressure()

        print(f"Temperature: {temperature:.2f} °C")
        print(f"Pressure: {pressure:.2f} hPa")
        print("-" * 30)
    #     time.sleep(2)

if __name__ == "__main__":
    main()

cheol904@cheol904:/opt/station $

```

Picture 5.6. Bmp280 sensor testing script.

After updating the script, both Python test programs were executed to read data from the sensors. The system returned valid readings for brightness (lux), temperature (°C), and pressure (hPa), confirming:

Correct module placement on the PCB

Proper PCB routing

Successful I2C communication with both devices

Functional sensor output after script modification

```

cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 0.0 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 544.1666666666667 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 3540.0 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 3070.0 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 3020.8333333333335 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 2577.5 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 2742.5 lux
cheol904@cheol904:/opt/station $ python3 test-bh1750.py
Light Level: 2682.5 lux
cheol904@cheol904:/opt/station $

```

Picture 5.7. Bh1750 sensor testing.

```

cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 30.10 °C
Pressure: 993.64 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.82 °C
Pressure: 993.63 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.69 °C
Pressure: 993.66 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.50 °C
Pressure: 993.68 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.43 °C
Pressure: 993.68 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.31 °C
Pressure: 993.66 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.25 °C
Pressure: 993.67 hPa
-----
cheol904@cheol904:/opt/station $ venv/bin/python3 test-bmp280.py
Temperature: 29.14 °C
Pressure: 993.67 hPa
-----
cheol904@cheol904:/opt/station $ █

```

Picture 5.8. Bmp280 sensor testing.

Materials Used

- BH1750 light sensor module
- BMP280 pressure/temperature sensor module
- Raspberry Pi
- Custom PCB (PIF 25/26)
- Socket strips with cables
- Soldering iron and solder

A6 ASSEMBLY

Task Description

The task involved assembling the station by preparing the wooden housing, drilling openings for the button, sensors, and power connector, installing the Raspberry Pi, PCB, and display, soldering and mounting the push button, attaching the BH1750 and BMP280 sensors to the rear panel, and routing all wiring so the components fit securely inside the enclosure and were ready for testing.

Part 1: Housing Assembly

The first stage consisted of assembling and preparing the wooden housing for the station. The housing is composed of three wooden wall elements and a removable rear panel.

The following steps were completed:

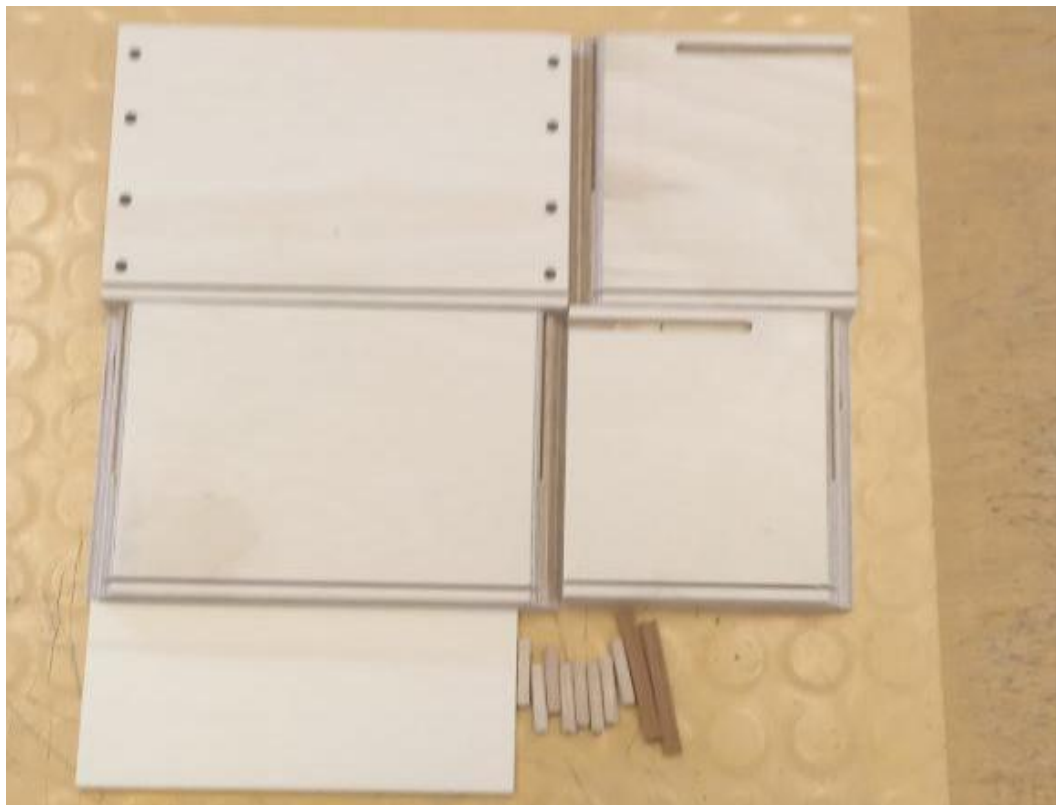
- All wooden parts were inspected for defects and aligned according to the work order.
- A circular hole was drilled in the left side wall to accommodate the push button. The position followed the specifications from the work order to ensure proper accessibility.
- Three pairs of holes were drilled in the rear panel: two pairs for mounting the BH1750 and BMP280 sensors, $\varnothing 3\text{mm}$ each, and one additional $\varnothing 4\text{mm}$ pair for attaching the power connector. All holes were positioned to ensure correct spacing and compatibility with the electronic components.
- The three housing walls were assembled and checked for correct fit. The rear sliding panel was inserted to verify proper tolerances.

Part 2: Electronics Installation

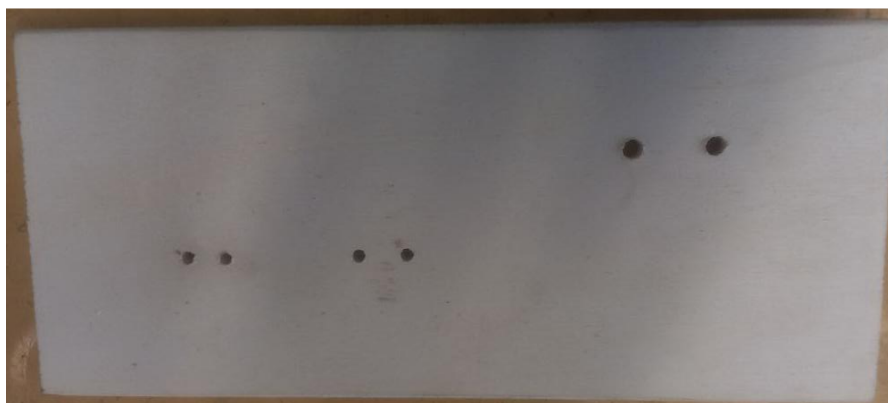
The second stage involved installing the electronic components inside the prepared housing and connecting all required wiring.

The following steps were completed:

- Wires were soldered to the push button to prepare it for installation. The solder joints were checked for mechanical stability and proper electrical contact.
- All electronic components, including the Raspberry Pi, the custom PCB, and the display, were installed inside the housing according to the work order.
- The push button was mounted into the drilled opening on the left wall and connected to the PCB using the previously soldered wires.
- The BH1750 light sensor and the BMP280 pressure/temperature sensor were attached to the rear panel using the mounting holes prepared in Part 1. Both sensors were connected to the PCB using the pre-soldered connectors.



Picture 6.1. Part of wooden housing



Picture 6.2. Drilled holes on rear panel



Picture 6.3. Drilled hole for button



Picture 6.4. Button with soldered wires

A7 LIGHTING

Task description

The task consisted of installing an LED lighting module inside the station. This included preparing the LED strip, routing the power wires through the housing, connecting the strip to the PCB, and mechanically attaching the LED strip to the interior of the station.

LED strip preparation

The LED strip was prepared for installation by soldering two power wires to its input pads. A red wire was connected to the 5V pad, and a black wire to the GND pad. After soldering, a piece of heat-shrink tubing was placed over the joints and heated to provide insulation and mechanical protection.

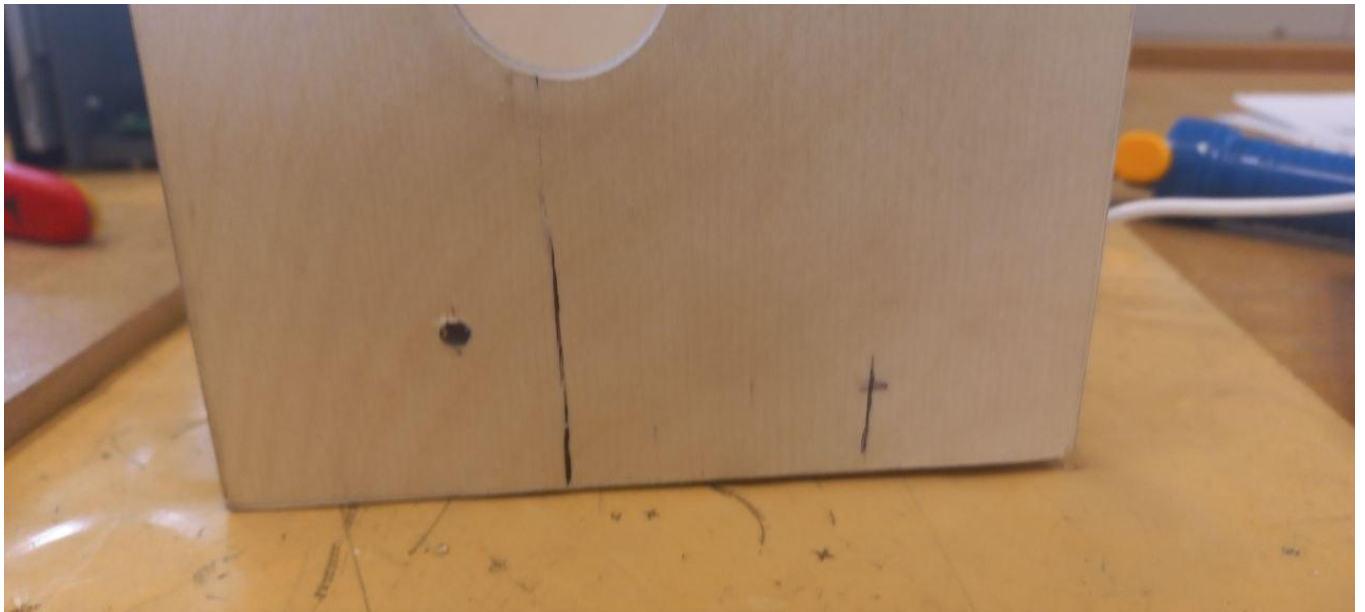


Picture 7.1. LED strip with wires soldered

Housing preparation

To route the power wires of the LED strip into the interior of the station, a 3 mm hole was drilled in the left wooden wall of the housing. The position of the opening followed the specifications from the work order to ensure that the wiring would not interfere with other components inside the enclosure. After drilling, the edges of the hole were cleaned to prevent damage to the insulation of the wires during installation.

The previously prepared wires, were then passed through the opening from the outside to the inside of the housing.



Picture 7.2. 3mm hole for wires

Connecting to PCB

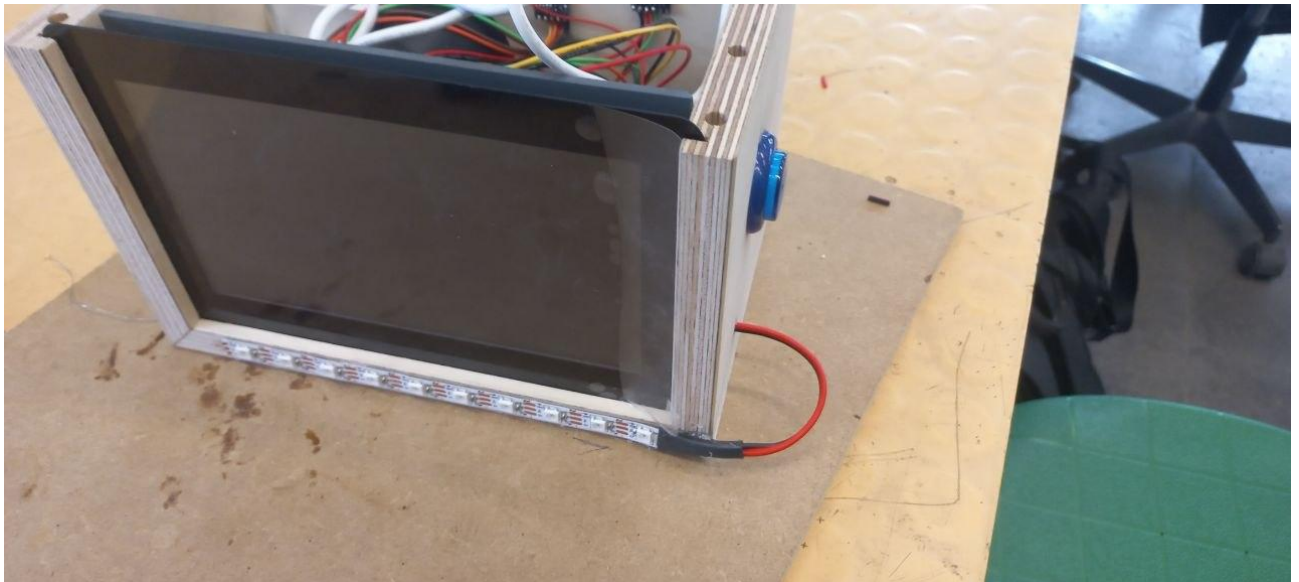
Inside the housing, the power wires from the LED strip were routed to the PCB and connected to the designated power input. The connection was made using a 2-pin screw terminal block, ensuring a secure and serviceable attachment. After tightening the terminal screws, the wiring was checked to confirm that both the 5V and GND lines were correctly positioned and firmly held in place. This completed the electrical integration of the LED strip into the station's power system.



Picture 7.3. Wires connected to screw terminal block

LED strip mounting

The LED strip was mounted on the station using a hot-glue gun. A thin, even layer of adhesive was applied along the back of the strip to ensure reliable contact with the wooden surface. After positioning, the strip was pressed firmly into place until the glue set. The installation was then inspected to confirm that the strip was aligned correctly, securely attached, and free of tension on the power wires.



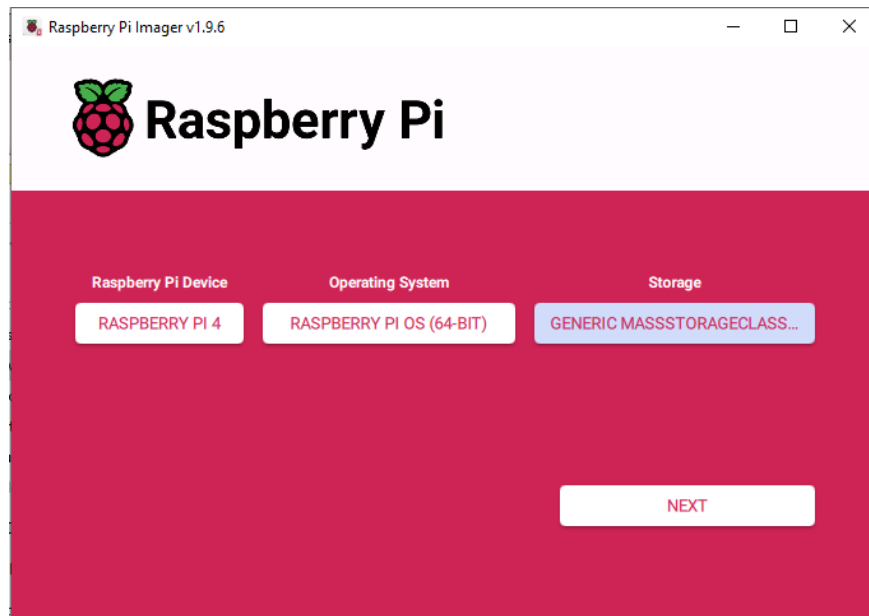
Picture 7.4. Mounted LED strip

L1 Raspberry Pi

Install and Configure

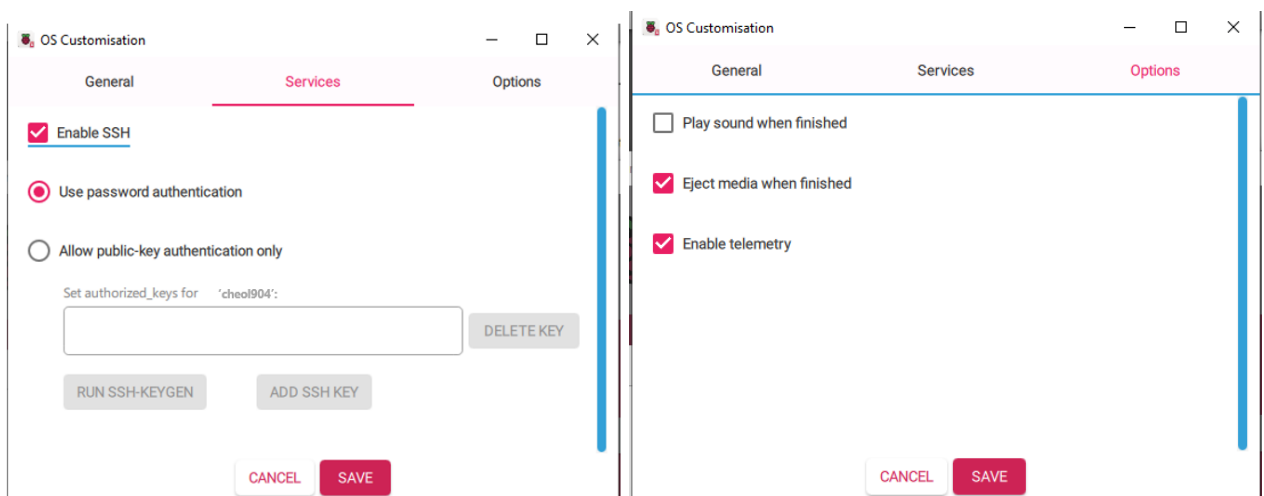
Install "Raspberry Pi OS (64-bit)" on the microSD card.

The operating system *Raspberry Pi OS (64-bit)* was installed on a microSD card by means of the Raspberry Pi Imager. After the installation process had been completed, the card was inserted into the Raspberry Pi and the system was successfully booted.



Picture 8.1. Raspberry Pi Imager interface

During the installation procedure, a hostname linked to the IAM system was specified, and a user account with a password was created. The Wi-Fi credentials of the laboratory network were entered, the time zone was set to the local region, and the keyboard layout was adjusted to the required language. SSH access was enabled to allow remote connections.



Pictures 8.2a and 8.2b. Services and Options

The image shows the 'OS Customisation' window with the 'General' tab selected. The window has three tabs: 'General', 'Services', and 'Options'. Under 'General', there are several sections:

- Set hostname:** A checkbox is checked, and the hostname is set to 'cheol904.local'.
- Set username and password:** A checkbox is checked. The username is 'cheol904' and the password is masked with dots.
- Configure wireless LAN:** A checkbox is checked. The SSID is 'LPEM_B13' and the password is masked with dots. There is an unchecked checkbox for 'Hidden SSID'.
- Wireless LAN country:** A dropdown menu is set to 'LU'.
- Set locale settings:** A checkbox is checked. The time zone is 'Europe/Luxembourg' and the keyboard layout is 'gb'.

At the bottom, there are 'CANCEL' and 'SAVE' buttons.

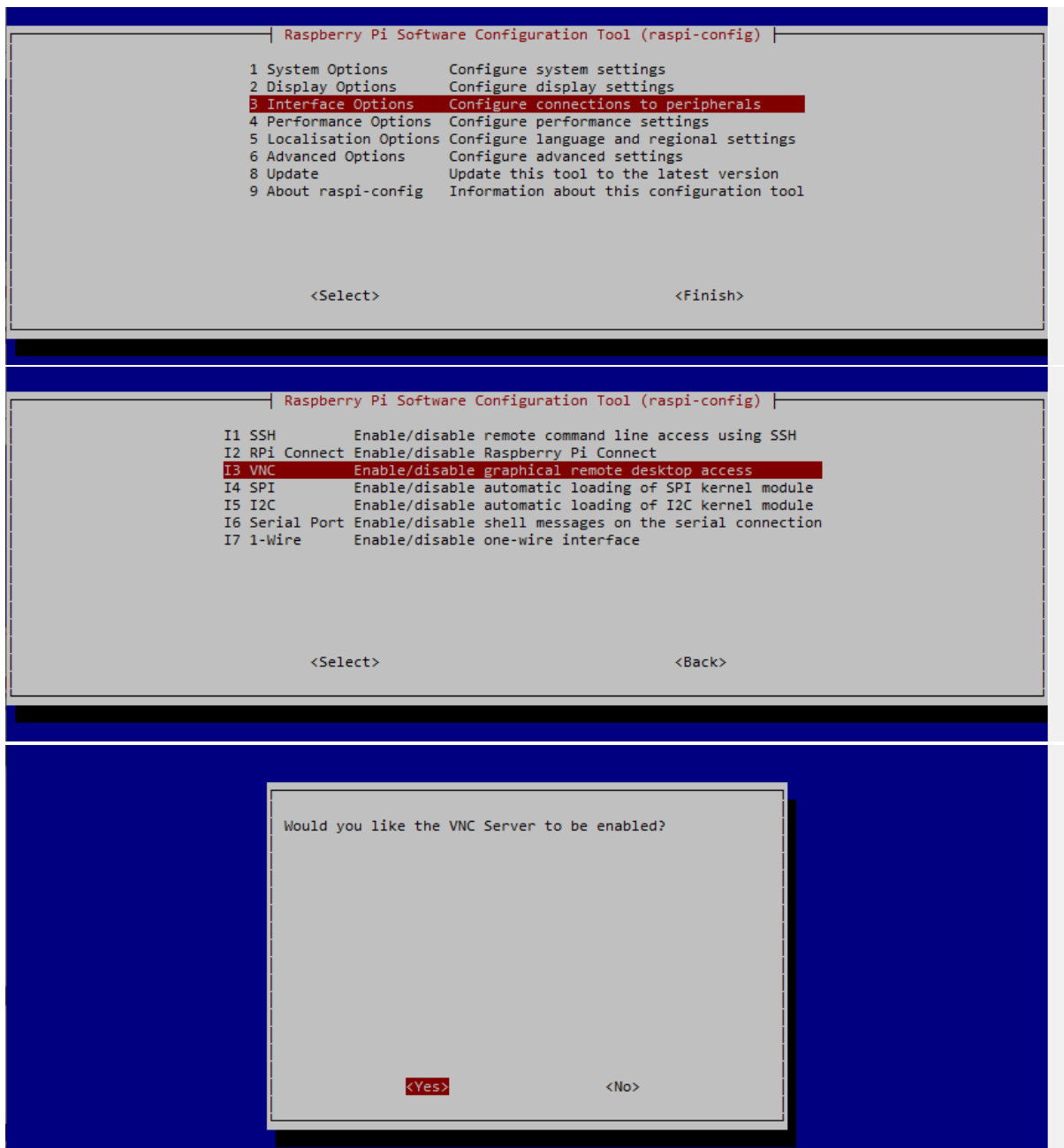
Picture 8.3. General configuration

Configure VNC

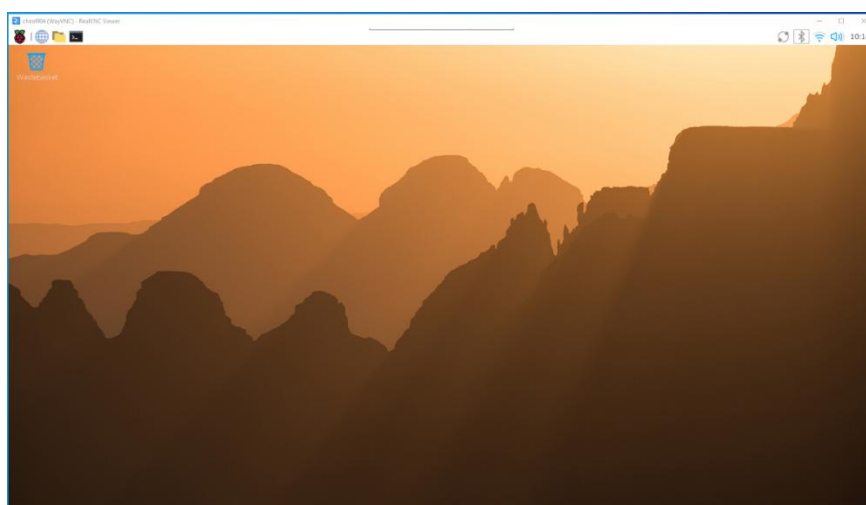
After connecting via SSH to raspberry PI the command

```
sudo raspi-config
```

was used to enter to the configuration tool and the VNC option was enabled. Once the service had been activated, the graphical interface of the Raspberry Pi became accessible from a remote machine.



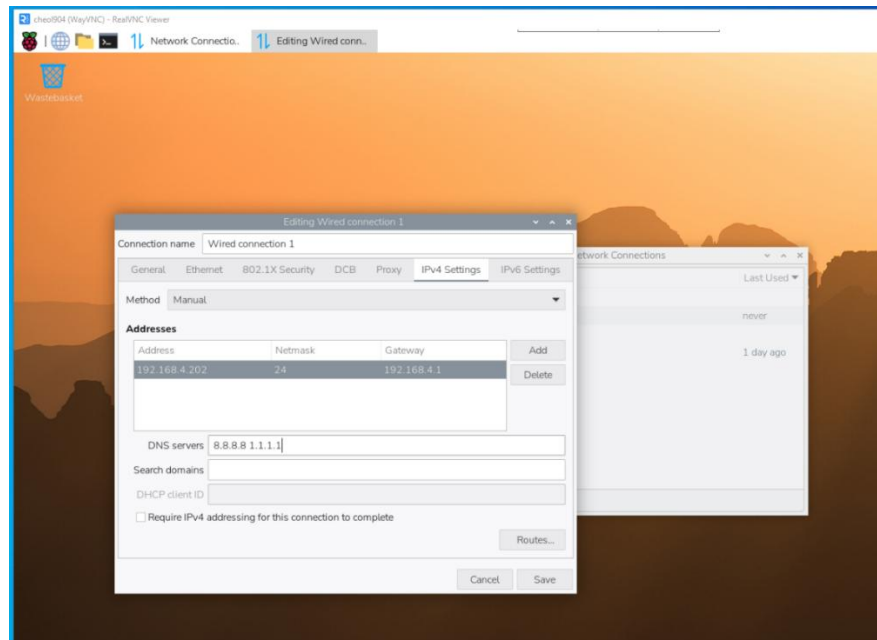
Pictures 8.4(a-c). Configuration of VNC



Picture 8.5. Remote connection established via VNC

Configure static IP for Ethernet

The static IP address was assigned through the graphical interface of the Raspberry Pi. The network settings window was opened, and the Ethernet interface was selected. In the configuration menu, the option for manual addressing was chosen. The IP address, subnet mask, gateway, and DNS server were entered into the corresponding fields. Ip was chosen as desk number + 200. After the parameters had been confirmed, the settings were saved, and the system was restarted.



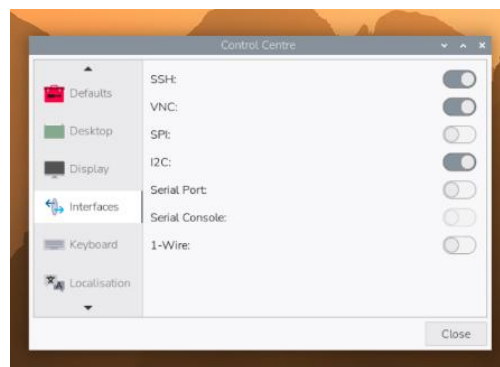
Picture 8.6. Network settings window

```
cheol1904@cheol1904:~$ hostname -I
192.168.4.202 192.168.4.90
cheol1904@cheol1904:~$
```

Picture 8.7. Verification of static IP

Enabling I²C

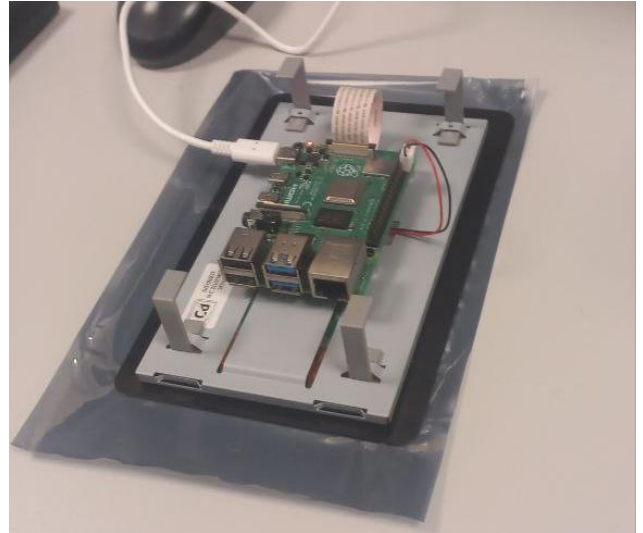
The I2C interface was enabled through the graphical configuration tool. The Raspberry Pi Configuration window was opened, and the “Interfaces” tab was selected. In the list of available options, the I2C function was switched on. After the change had been confirmed, the system was restarted.



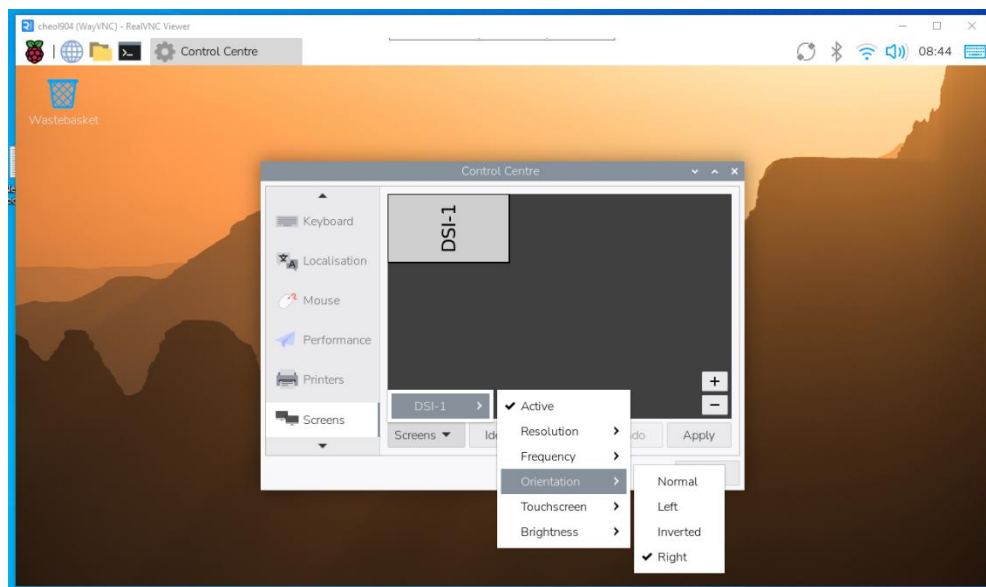
Picture 8.8 Enabling the I2C option.

Install and configure screen

The display was connected to the Raspberry Pi and detected automatically during startup. In the graphical settings menu, the orientation parameter was adjusted, and the option “Right” was selected. After the change had been confirmed, the configuration was saved and the system was restarted.



Pictures 8.9a and 8.9b. Touchscreen display connected



Picture 8.10. Screen orientation set to “Right” in Control Centre

Docker

Docker installation

Docker was installed via terminal using the official installation script. The following commands were executed:

```
sudo apt update && sudo apt upgrade -y  
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh  
sudo usermod -aG docker $USER  
newgrp docker
```

Firmware and website files

Folders /opt/station and /opt/station/www were created to store firmware and website files:

```
sudo mkdir -p /opt/station/www  
sudo chown -R root:root /opt/station/www  
sudo chmod -R 755 /opt/station/www
```

All firmware files were moved to /opt/station. All website files were moved to /opt/station/www.

```

cheol904@cheol904:/opt/station $ ls -l
total 40
drwxr-xr-x 3 root root 4096 Nov 19 15:02 bh1750
drwxr-xr-x 4 root root 4096 Nov 19 15:02 bme280
-rw-r--r-- 1 root root 551 Nov 19 15:02 config.toml
-rw-r--r-- 1 root root 6043 Nov 19 15:02 measure.py
drwxr-xr-x 3 root root 4096 Nov 19 15:02 mq135
-rw-r--r-- 1 root root 409 Nov 19 15:02 test-bh1750.py
-rw-r--r-- 1 root root 585 Nov 19 15:02 test-bme280.py
-rw-r--r-- 1 root root 400 Nov 19 15:02 test-mq135.py
drwxr-xr-x 2 root root 4096 Nov 19 15:02 www
cheol904@cheol904:/opt/station $ ls -l bh1750
total 8
-rw-r--r-- 1 root root 506 Nov 19 15:02 __init__.py
drwxr-xr-x 2 root root 4096 Nov 19 15:02 __pycache__
cheol904@cheol904:/opt/station $ ls -l bh1750/__pycache__/
total 4
-rw-r--r-- 1 root root 1183 Nov 19 15:02 __init__.cpython-311.
cheol904@cheol904:/opt/station $ ls -l bme280/
total 24
drwxr-xr-x 3 root root 4096 Nov 19 15:02 i2cdevice
-rw-r--r-- 1 root root 372 Nov 19 15:02 __init__.py
-rw-r--r-- 1 root root 10536 Nov 19 15:02 lib.py
drwxr-xr-x 2 root root 4096 Nov 19 15:02 __pycache__
cheol904@cheol904:/opt/station $ ls -l bme280/__pycache__/
total 20
-rw-r--r-- 1 root root 1310 Nov 19 15:02 __init__.cpython-311
-rw-r--r-- 1 root root 15475 Nov 19 15:02 lib.cpython-311.pyc
cheol904@cheol904:/opt/station $ ls -l bme280/i2cdevice/
total 20
-rw-r--r-- 1 root root 1283 Nov 19 15:02 adapter.py
-rw-r--r-- 1 root root 9299 Nov 19 15:02 __init__.py
drwxr-xr-x 2 root root 4096 Nov 19 15:02 __pycache__

```

```

-rw-r--r-- 1 root root 9299 Nov 19 15:02 __init__.py
drwxr-xr-x 2 root root 4096 Nov 19 15:02 __pycache__
cheol904@cheol904:/opt/station $ ls -l bme280/i2cdevice/__pycache__/
total 24
-rw-r--r-- 1 root root 3349 Nov 19 15:02 adapter.cpython-311.pyc
-rw-r--r-- 1 root root 16528 Nov 19 15:02 __init__.cpython-311.pyc
cheol904@cheol904:/opt/station $ ls -l mq135/
total 32
-rw-r--r-- 1 root root 912 Nov 19 15:02 __init__.py
-rw-r--r-- 1 root root 21607 Nov 19 15:02 lib.py
drwxr-xr-x 2 root root 4096 Nov 19 15:02 __pycache__
cheol904@cheol904:/opt/station $ ls -l mq135/__pycache__/
total 28
-rw-r--r-- 1 root root 1367 Nov 19 15:02 __init__.cpython-311.pyc
-rw-r--r-- 1 root root 21964 Nov 19 15:02 lib.cpython-311.pyc
cheol904@cheol904:/opt/station $ ls -l www
total 4
-rw-r--r-- 1 root root 2204 Nov 19 15:02 index.php

```

Picture 8.11a and 8.11b. Firmware and Website files

Config.toml file configuration

The configuration file /opt/station/config.toml was adjusted according to the work order.

```
cheol904@cheol904:~ $ cat /opt/station/config.toml
```

```
#station_serial = "cheol904"           # Serial number of the station

interval = 2                           # Time between measurements

toggle_intensity = 100                  # The light level at which the LED should be turned on


[i2c]                                   # I2C-Addresses of the sensors
#bme280 = ""
#bh1750 = ""
#mq135 = ""


[destination]                           # Information about the server
#ip = "127.0.0.1"                       # IP-Address or Hostname of the Server
#path = "/"                             # Path to the script that accepts the measurements
```

Creating Docker container

A Docker container was created to host the website using the official PHP image with Apache. The container was configured to restart automatically and included volume bindings for firmware and web files. Port 8080 was mapped to allow access via browser. The following command was executed:

```
docker run -d --name php --restart unless-stopped -p 8080:80 -v
/tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

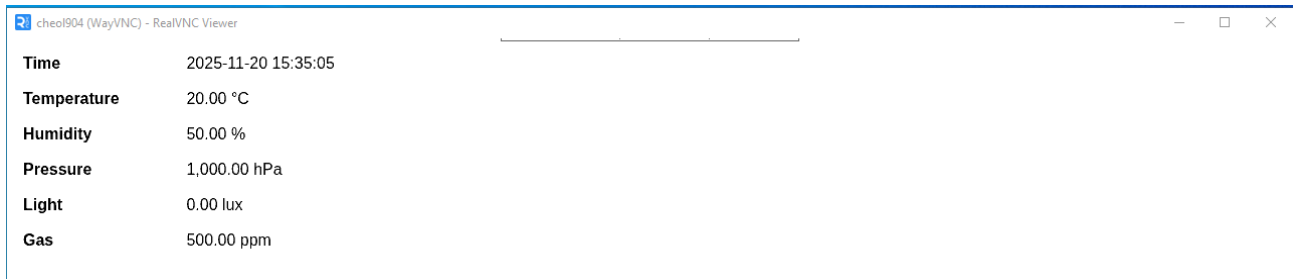
```
cheol904@cheol904:~ $ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
78d1dd83b3e4   php:8.4-apache "docker-php-entrypoi..." 24 hours ago   Up 3 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php
```

Picture 8.12 Container php.

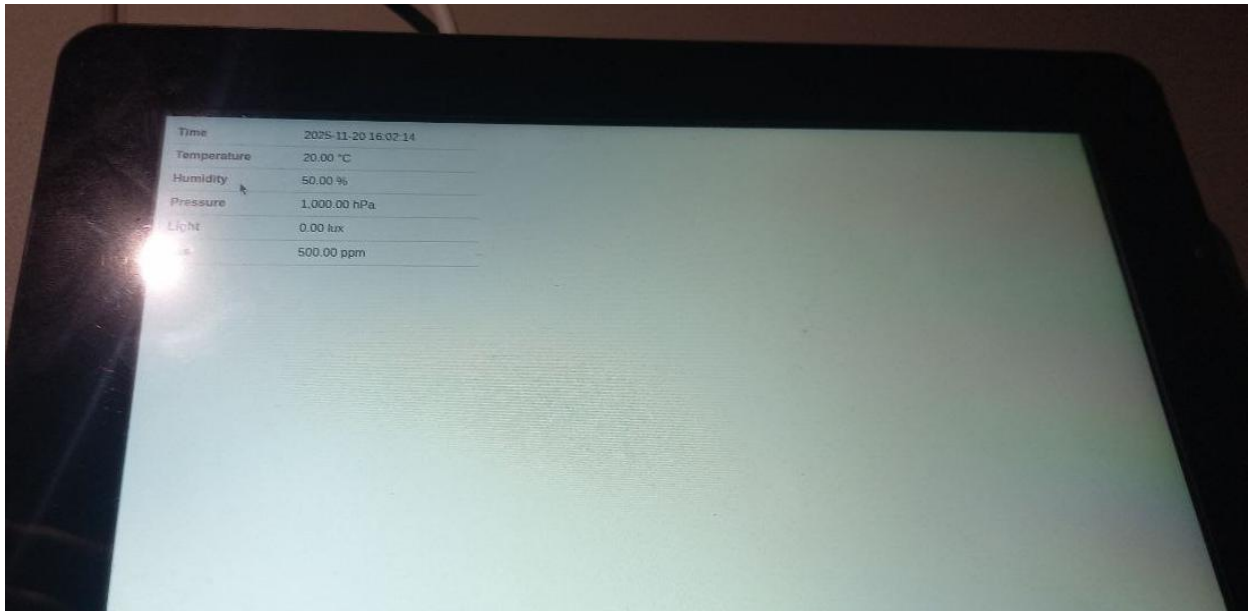
Configure autostart

The autostart file was edited to ensure that the measurement script, Docker service, and the website in kiosk mode would start automatically after boot. The following commands were added to `~/config/labwc/autostart`:

```
mkdir -p /tmp/station
python3 /opt/station/measure.py --offline > /tmp/station/result.csv &
sudo systemctl start docker
docker start php
sleep 5
chromium http://localhost:8080 \
--kiosk --noerrdialogs --disable-infobars --no-first-run \
--enable-features=OverlayScrollbar --start-maximized &
```



Picture 8.13. Website opened in Chromium



Picture 8.14. Website opened in Chromium kiosk mode

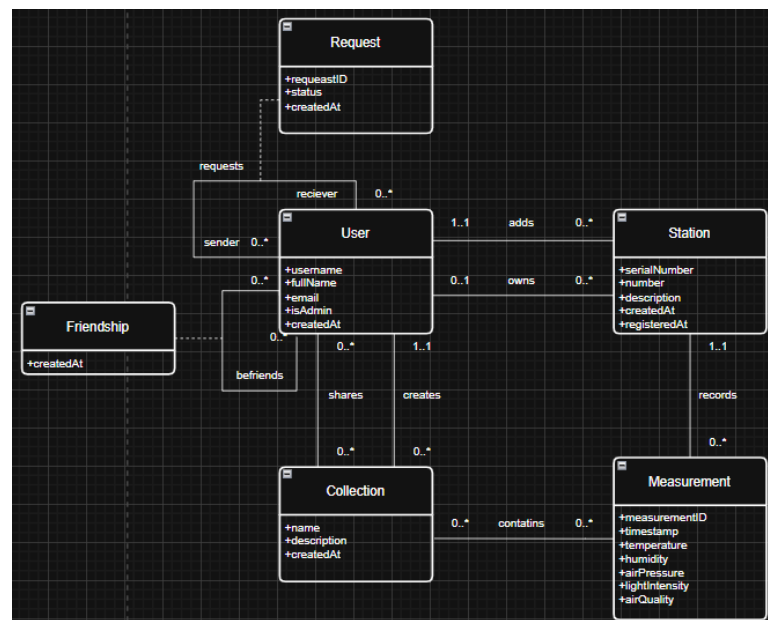
Resources

1. <https://www.raspberrypi.com/documentation/computers/configuration.html#ras-pi-config>
2. <https://docs.docker.com/engine/install/debian/>
3. <https://docs.docker.com/engine/install/linux-postinstall/#manage-docker-as-a-non-root-user>
4. <https://docs.docker.com/engine/storage/bind-mounts/>
5. <https://www.raspberrypi.com/tutorials/how-to-use-a-raspberry-pi-in-kiosk-mode/>

L2 DATABASE

Conceptual Data Model (CDM)

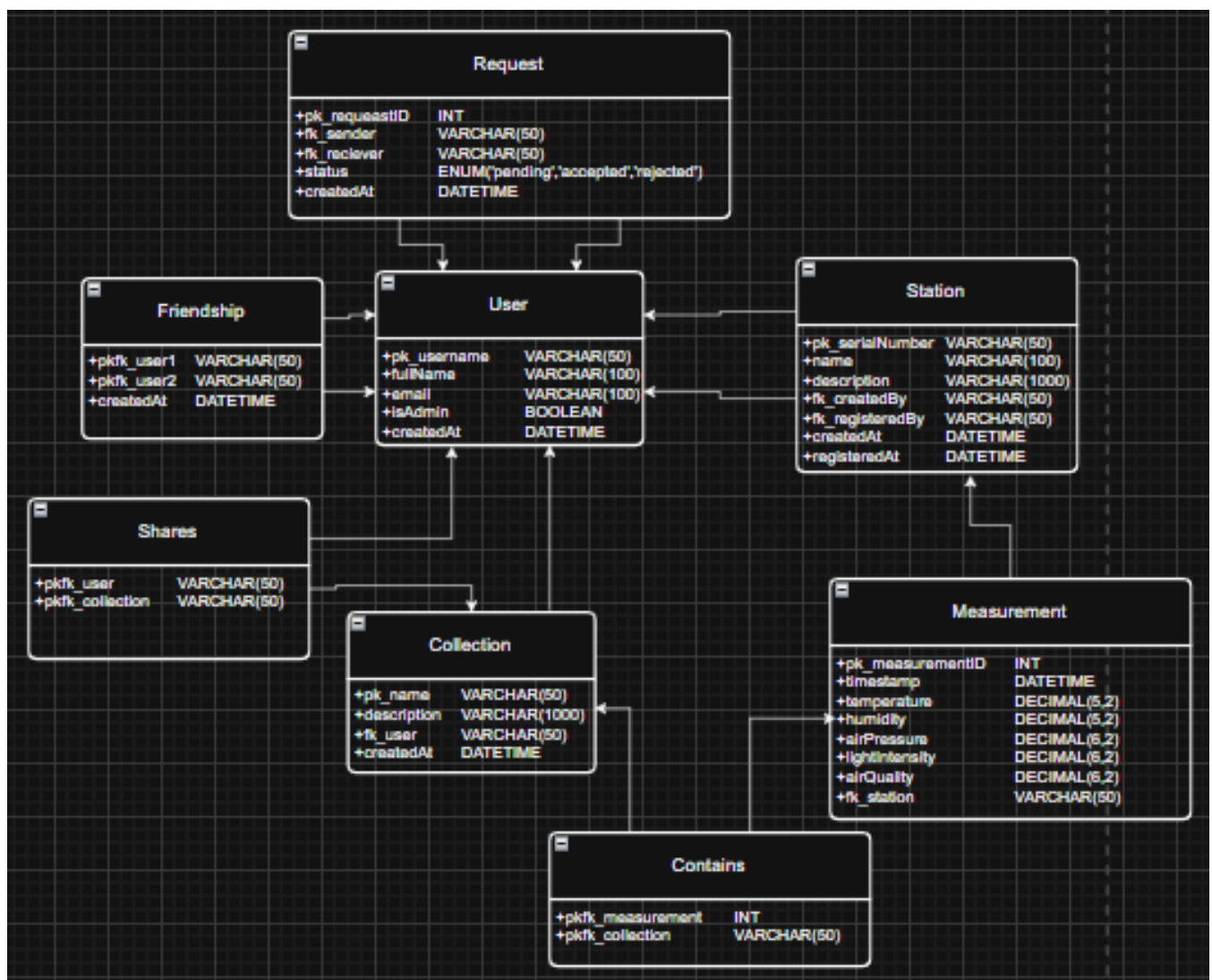
The Conceptual Data Model (CDM) provides a high-level representation of the entities and relationships within the system. It abstracts away implementation details and focuses instead on the logical organization of data. The CDM illustrates how users, requests, friendships, stations, measurements, collections, and sharing mechanisms interact conceptually. This model serves as the foundation for subsequent logical and physical designs, ensuring that the database structure aligns with the intended functional requirements of the application.



Picture 9.1. Conceptual Data Model

Logical Data Model (LDM)

The Logical Data Model (LDM) refines the conceptual design into a more detailed schema that specifies attributes, primary keys, and foreign keys. It translates abstract relationships into precise constraints and data types, thereby bridging the gap between conceptual understanding and physical implementation. The LDM ensures normalization, enforces referential integrity, and defines the cardinality of relationships. This stage is critical for preventing redundancy, maintaining consistency, and preparing the schema for direct translation into SQL code.

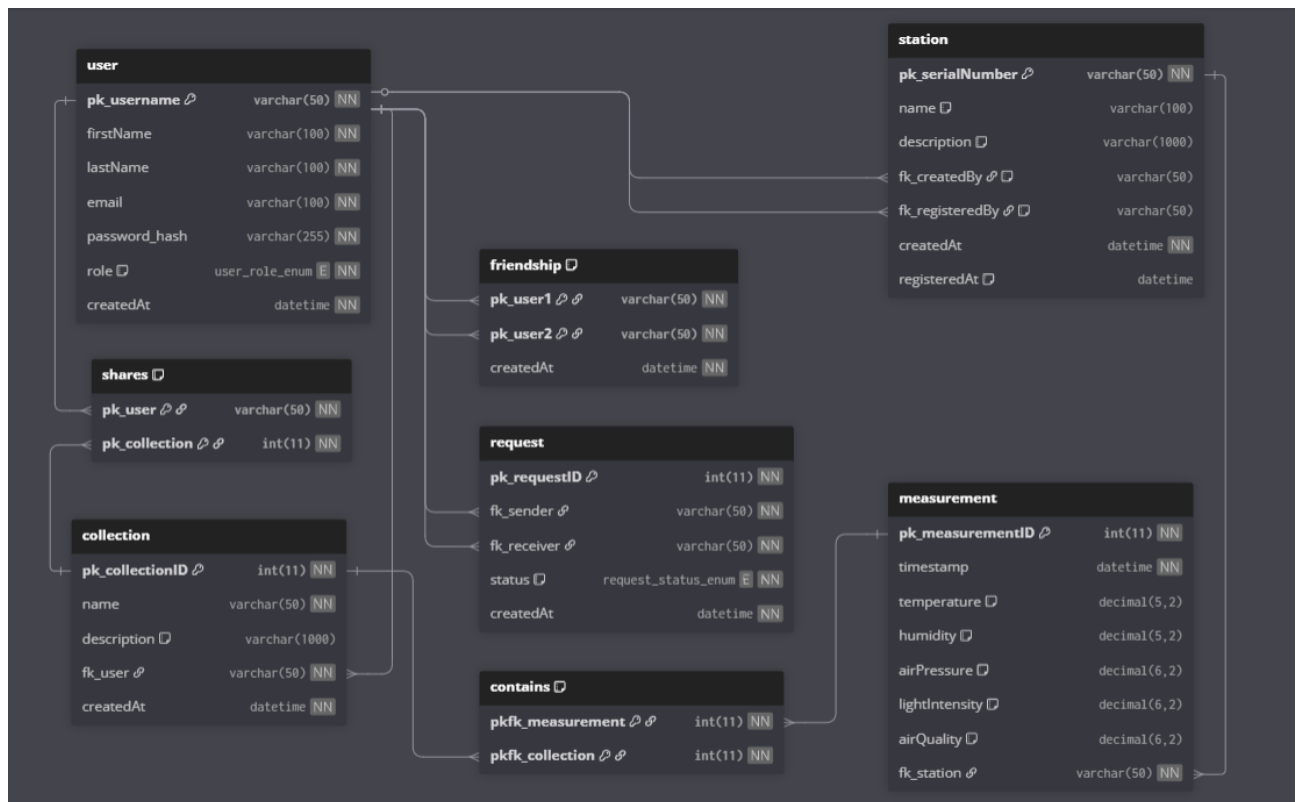


Picture 9.2. Logical Data Model

Creating Database

The database creation phase translates the logical data model into a physical schema using SQL. This stage establishes the structural backbone of the system by defining tables, attributes, constraints, and relationships. Each table is designed to ensure referential integrity, data consistency, and scalability.

- **User Table:** Defines the core entity of the system. Each user is uniquely identified by a primary key (pk_username). Attributes include full name, email, administrative privileges, and creation timestamp. Constraints enforce uniqueness and guarantee that essential information is not null.
- **Request Table:** Represents friendship requests between users. Each request is assigned an auto-incremented identifier (pk_requestID). Foreign keys link sender and receiver to the User table, while the status attribute is constrained by an enumeration (pending, accepted, rejected). This ensures controlled states for request processing.
- **Friendship Table:** Records established friendships. The composite primary key (pk_user1, pk_user2) prevents duplication, while a check constraint enforces ordering ($\text{pk_user1} < \text{pk_user2}$) to maintain consistency. Foreign keys guarantee that both participants exist in the User table.
- **Station Table:** Stores metadata about sensor stations. Each station is uniquely identified by a serial number. Attributes include name, description, creator, registrar, and timestamps for creation and registration. Referential integrity is enforced by linking creator and registrar to the User table.
- **Measurement Table:** Contains sensor readings associated with stations. Each measurement is uniquely identified by an auto-incremented primary key. Attributes capture environmental parameters such as temperature, humidity, air pressure, light intensity, and air quality. A foreign key links each measurement to its corresponding station.
- **Collection Table:** Represents user-defined groupings of measurements. Each collection has a unique name, descriptive metadata, ownership information, and a creation timestamp. Referential integrity is maintained by linking collections to their creators in the User table.
- **Shares Table:** Manages collaborative access to collections. The composite primary key (pk_user, pk_collection) ensures uniqueness of sharing relationships. Foreign keys enforce valid references to both users and collections.
- **Contains Table:** Establishes a many-to-many relationship between measurements and collections. The composite primary key (pkfk_measurement, pkfk_collection) prevents duplication, while foreign keys ensure that only valid measurements and collections are linked.



Picture 9.3.Updated Logical Data Model

L3 WEBSITE

Introduction

The WetherStation platform is a sophisticated web-based ecosystem designed to bridge the gap between IoT data management and social collaboration. The application provides users with powerful tools to monitor hardware stations, visualize telemetry data, and interact with other community members through integrated communication modules. Built with a focus on security, scalability, and user experience, the platform utilizes modern frameworks to ensure seamless operation across all device types.

Authentication and Security

Login and Authentication

The login module serves as the primary gateway to the system, ensuring secure access to user data. The system performs a rigorous verification of credentials, matching the provided username and password against encrypted records in the database. Upon successful authentication, the server initiates a secure session and assigns a specific User Role (User or Administrator), which dictates the scope of accessible features and data.

Registration and Email Verification

New users must undergo a structured registration process to maintain the integrity of the community. The system validates the uniqueness of the chosen username and email address to prevent duplicate accounts. To ensure the validity of the contact information, a verification code is dispatched via SMTP to the user's email; account creation is only finalized once the correct code is submitted, thereby mitigating the risk of bot registrations.

User Workspace

Personal Dashboard

The Dashboard provides a centralized, high-level overview of the user's entire ecosystem. It aggregates key statistics, including the total count of registered stations, active friendships, curated collections, and historical measurements. Furthermore, the interface displays the most recent telemetry data from each connected station, allowing for immediate status monitoring without navigating through sub-menus.

Profile Management and Customization

The Profile section empowers users to manage their identity and interface preferences. Users can update personal details such as full names and email addresses, or perform secure password resets to maintain account safety. Personalization options include the ability to select from a gallery of default avatars or upload custom images, alongside settings for the preferred system language and visual theme.

Public Profile View

To facilitate social discovery, the platform includes a simplified Public Profile View accessible to other registered members. This view hides sensitive account settings and private information, displaying only the user's basic identity and public-facing activity. It serves as the primary interface for users to evaluate potential friendship connections and view shared contributions.

IoT and Data Management

Station Management

The Stations module provides a comprehensive directory of both currently active (registered) devices and historical hardware associated with the account. Users are granted the authority to modify station metadata, such as titles and descriptive notes, to better organize their hardware fleet. New devices are integrated into the system via a secure registration process requiring a unique, hardware-generated synchronization code.

Measurement Visualization and Analytics

The Measurements tab is a robust tool for data analysis, providing access to all telemetry recorded by the user's stations. Users can apply granular filters based on specific collections, stations, or custom date ranges to isolate relevant datasets. Data can be toggled between structured tabular

views and dynamic graphical charts, with support for exporting information into CSV (for raw data) or PNG (for visual charts) formats.

Collaborative Features

Collections and Data Sharing

Collections allow for the modular grouping of data through a "slot" system, where each slot represents a specific station paired with a defined timeframe. The "My Collections" tab enables the creation of new groups with custom descriptions, which can then be shared with specific friends for collaborative analysis. The system includes built-in logic to prevent time-frame overlaps for the same station, ensuring data consistency and logical integrity during the editing process.

Social Integration (Friends)

The Friends module facilitates networking between users through a secure request-and-accept workflow. Users can utilize a global search function to locate others by username and dispatch friendship invitations. Once a request is accepted, a formal link is established, granting the users the ability to share collections and participate in group communication.

Real-Time Communication (Chat)

The Chat system supports both private one-on-one dialogues and multi-user group conversations. The module supports rich media exchange, with a validation engine that checks file size, extension, and MIME type to ensure security and compatibility. For enhanced user experience, a persistent "Draft" feature is implemented, allowing users to leave a conversation and return to find their unsent messages intact.

System Administration and UI

Administrator Control Panel

Restricted to users with elevated privileges, the Admin Panel provides full CRUD (Create, Read, Update, Delete) capabilities across the entire database. It is divided into five specialized sectors: Users, Stations, Measurements, Collections, and Posts. The "Posts" sub-module acts as a Content Management System (CMS), allowing admins to broadcast system-wide notifications with configurable visibility settings for specific user groups.

Notification System (Bell Icon)

The notification center serves as an asynchronous alert hub for all account-related activity. It provides real-time updates regarding incoming friendship requests, collection sharing invitations, and administrative announcements. This ensures that users remain informed of critical events without needing to manually refresh different sections of the application.

Localization and Responsive Design

To accommodate a diverse user base, the platform offers full localization in English, French, and Ukrainian, which can be toggled instantly via the language selector. The visual experience is further

refined by a theme-switching mechanism (Light/Dark mode). The entire interface is built upon the Bootstrap framework, ensuring a fully responsive layout that maintains professional aesthetics and functionality on desktops, tablets, and mobile devices.

L4 WEB-DATABASE SERVER

Introduction

The purpose of this sprint was to design, deploy, and configure a complete Web-Database Server using Docker technologies. The system integrates a PHP-based web server, a MariaDB relational database, phpMyAdmin for database administration, and Mailhog for email testing.

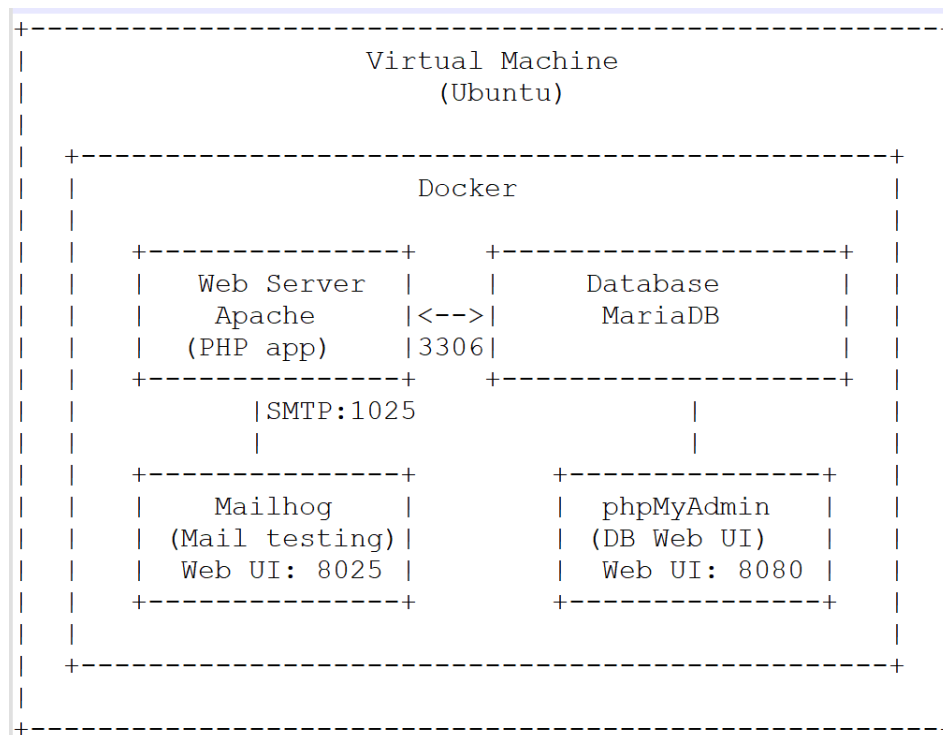
System Architecture

The system developed in this sprint is based on a modular, container-oriented architecture using Docker Compose. Each service runs in its own isolated container while communicating through an internal Docker network automatically created by Docker Compose.

The architecture consists of four main components:

- Web Server (Apache + PHP 8.2) — responsible for serving the website and executing PHP scripts.
- MariaDB Database Server — stores all persistent application data and is accessible only to authorized services.
- phpMyAdmin — provides a graphical interface for managing the MariaDB database.
- Mailhog — captures outgoing emails for testing purposes and provides a web interface for viewing them.

All services communicate using service names instead of IP addresses, which simplifies configuration and improves reliability. The corrected architecture diagram is shown below.



Picture 11.1. Project architecture

Environment and Docker Installation

The system was deployed on a virtual machine running Ubuntu Server 22.04 LTS. The installation process included configuring secure remote access, updating system packages, installing Docker Engine, and enabling Docker Compose support. These steps ensure that the environment is ready to host the multi-container architecture described in the previous section.

Virtual Machine Preparation

A clean installation of Ubuntu Server 22.04 LTS was performed on the virtual machine. After the initial setup, the system packages were updated to ensure the latest security patches and software versions were applied:

```
sudo apt update && sudo apt upgrade -y
```

SSH Installation and Configuration

To enable remote administration of the server, the OpenSSH Server package was installed:

```
sudo apt install openssh-server -y
```

After installation, the SSH service was enabled and started:

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

The service status was verified to ensure successful activation:


```
sudo systemctl status ssh
```

Docker Engine Installation

Docker Engine was installed using the official installation script provided by Docker. This method ensures that the latest stable version is installed:

```
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh
```

After installation, the current user was added to the docker group to allow running Docker commands without sudo:

```
sudo usermod -aG docker $USER  
newgrp docker
```

The installation was verified with:

```
docker --version  
docker run hello-world
```

Docker Compose Installation

Docker Compose v2 is included as part of modern Docker Engine distributions. To ensure it is available, the following package was installed:

```
sudo apt install docker-compose -y
```

The version was confirmed using:

```
docker compose version
```

Directory Structure Preparation

A dedicated project directory was created to store all configuration files, website files, SQL initialization scripts, and Apache VirtualHost definitions:

```
mkdir -p ~/repif  
cd ~/repif
```

This directory serves as the root of the entire deployment and contains all files required for the Docker Compose stack.

Project Structure

To organize all components of the web-database server, a dedicated project directory was created on the virtual machine.

```
repif/
├── apache/
│   └── vhosts/
│       └── 000-default.conf
├── db-init/
│   └── 01_init.sql
├── www/
│   └── public/
│       ├── index.php
│       └── dbtest.php
├── .env
└── docker-compose.yml
```

Picture 11.2. Project Structure

Description of Each Directory

- `apache/`: Contains Apache configuration files. The `vhosts/` subdirectory stores the VirtualHost definition used by the web server.
- `db-init/`: Contains SQL initialization scripts executed automatically when the MariaDB container is created. The file `01_init.sql` creates the database and the restricted user.
- `www/`: Contains all website files. The `public/` folder is used as the DocumentRoot for Apache. It includes:
 - `index.php` — main page of the website
 - `dbtest.php` — script for testing database connectivity
- `.env`: Stores environment variables such as database credentials, phpMyAdmin login, and other sensitive configuration values. This keeps the `docker-compose.yml` clean and secure.
- `docker-compose.yml`: Defines the entire multi-container application, including:
 - web server (Apache + PHP)
 - MariaDB database
 - phpMyAdmin interface
 - Mailhog email testing serviceIt also configures volumes, networks, ports, and restart policies.

Docker Compose Stack

To deploy the complete web-database server, a multi-container environment was created using Docker Compose. This approach allows all services—web server, database, phpMyAdmin, and Mailhog—to be started, stopped, and managed as a single unit. The configuration is stored in the file `docker-compose.yml` located in the project directory `~/repif`.

The stack consists of four containers:

- web — Apache + PHP 8.2 web server
- db — MariaDB 11 database
- phpmyadmin — graphical interface for database management
- mailhog — SMTP testing tool for capturing outgoing emails

All sensitive values (passwords, usernames, database names) are stored in the `.env` file to keep the configuration clean and secure.

`docker-compose.yml`

```
version: "3.9"

services:
  web:
    image: php:8.2-apache
    container_name: web
    ports:
      - "80:80"
    volumes:
      - ./www:/var/www/html
      - ./apache/vhosts:/etc/apache2/sites-available
    environment:
      DB_HOST: db
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
```

DB_PASS: \${DB_PASS}

depends_on:

- db

restart: unless-stopped

db:

image: mariadb:11

container_name: db

environment:

MYSQL_ROOT_PASSWORD: \${DB_ROOT_PASSWORD}

MYSQL_DATABASE: \${DB_NAME}

MYSQL_USER: \${DB_USER}

MYSQL_PASSWORD: \${DB_PASS}

volumes:

- db_data:/var/lib/mysql

- ./db-init:/docker-entrypoint-initdb.d

restart: unless-stopped

healthcheck:

test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]

interval: 5s

timeout: 3s

retries: 5

phpmyadmin:

image: phpmyadmin/phpmyadmin:latest

container_name: phpmyadmin

ports:

- "8080:80"

environment:

PMA_HOST: db

PMA_USER: \${PMA_USER}

PMA_PASSWORD: \${PMA_PASSWORD}

depends_on:
- db
restart: unless-stopped
mailhog:
image: mailhog/mailhog:latest
container_name: mailhog
ports:
- "8025:8025"
restart: unless-stopped
volumes:
db_data:

Explanation of the Stack

web (Apache + PHP)

- Serves the website from ./www
- Loads VirtualHost configs from ./apache/vhosts
- Connects to MariaDB using environment variables
- Depends on the database container

db (MariaDB 11)

- Initializes the database using scripts from ./db-init
- Stores data persistently in the db_data volume
- Provides credentials via .env

phpmyadmin

- Accessible at http://localhost:8080
- Connects to the database using PMA_HOST=db

mailhog

- Captures outgoing emails
- Web interface available at http://localhost:8025

Using the Stack

To start all services:

```
docker compose up -d
```

To stop:

```
docker compose down
```

To view logs:

```
docker compose logs -f
```

Environment Variables

To keep sensitive information secure and to maintain a clean and readable Docker Compose configuration, all credentials and service-specific parameters were stored in a separate `.env` file located in the project root directory `~/repif`. The `.env` file is automatically loaded by Docker Compose when executing any `docker compose` command.

`.env` File Contents

```
DB_ROOT_PASSWORD=rootpassword
```

```
DB_NAME=appdb
```

```
DB_USER=webuser
```

```
DB_PASS=webpassword
```

```
PMA_USER=webuser
```

```
PMA_PASSWORD=webpassword
```

Apache Configuration and VirtualHost

The web server is based on the official Docker image `php:8.2-apache`, which includes both Apache 2.4 and PHP 8.2. All Apache configuration is stored inside the Docker container, while the configuration files themselves are mounted from the project directory on the virtual machine.

The Apache configuration files are stored in:

```
~/repif/apache/vhosts/
```

These files are mounted into the container at:

```
/etc/apache2/sites-available
```

VirtualHost Configuration File

The main VirtualHost file used in this project is:

```
~/repif/apache/vhosts/000-default.conf
```

Full content of the file:

```
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/public

    <Directory /var/www/html/public>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Enabling Required Apache Modules

To support .htaccess and URL rewriting, the following Apache modules must be enabled inside the container:

```
docker exec -it web a2enmod rewrite
docker exec -it web a2enmod headers
```

After enabling the modules, Apache must be restarted:

```
docker exec -it web service apache2 restart
```

Activating the VirtualHost

The VirtualHost file must be enabled inside the container:

```
docker exec -it web a2ensite 000-default.conf
```

Then reload Apache:

```
docker exec -it web service apache2 reload
```

The default Apache site can be disabled:


```
docker exec -it web a2dissite 000-default.conf
```

Database Initialization and Security

The MariaDB database server is deployed as a Docker container and initialized automatically during its first startup. To ensure a secure and consistent database environment, an initialization script was created and placed in the `~/repif/db-init/`.

This script is executed automatically by the MariaDB entrypoint when the container is created for the first time. Additionally, a restricted-privilege database user was configured.

Database Initialization Script

The initialization script is stored in:

```
~/repif/db-init/01_init.sql
```

Below is the full content of the script:

```
CREATE DATABASE IF NOT EXISTS appdb;
CREATE USER IF NOT EXISTS 'webuser'@'%' IDENTIFIED BY 'webpassword';
GRANT SELECT, INSERT, UPDATE, DELETE ON appdb.* TO 'webuser'@'%';
FLUSH PRIVILEGES;
```

Running `mysql_secure_installation`

Although MariaDB inside Docker is already configured with a root password (from `.env`), it is still recommended to run `mysql_secure_installation` to apply additional security settings.

This can be done inside the running container:

```
docker exec -it db mysql_secure_installation
```

Typical recommended settings:

- Remove anonymous users: Yes
- Disallow root login remotely: Yes
- Remove test database: Yes
- Reload privilege tables: Yes

PHP Example Application

To verify the correct operation of the web server, PHP interpreter, and database connectivity, a minimal example application was created inside the project directory. The application consists of two PHP files:

index.php — the main landing page

dbtest.php — a script used to test the connection between PHP and MariaDB

Both files are located in:

~/repif/www/public/

This directory corresponds to the Apache DocumentRoot defined in the VirtualHost configuration.

index.php

The index.php file serves as the main entry point of the website and confirms that the Apache + PHP container is functioning correctly.

```
<?php
echo "<h1>Welcome to the Web-Database Server</h1>";
echo "<p>Your PHP environment is working correctly.</p>";
?>
```

Welcome to the Web-Database Server

Your PHP environment is working correctly.

Picture 11.3. Apache + Php container works

dbtest.php

The dbtest.php script is used to verify that the PHP application can successfully connect to the MariaDB database using the credentials defined in the .env file.

```
<?php
$host = getenv('DB_HOST');
$db   = getenv('DB_NAME');
$user = getenv('DB_USER');
$pass = getenv('DB_PASS');
```

```
$conn = mysqli_connect($host, $user, $pass, $db);  
if (!$conn) {  
    echo "<h2>Database connection failed:</h2>";  
    echo mysqli_connect_error();  
    exit;  
}  
echo "<h2>Database connection successful!</h2>";  
mysqli_close($conn);  
?>
```

Database connection successful!

Picture 11.4. Database connected

Mailhog Configuration and Email Testing

Mailhog is a lightweight SMTP testing tool used to capture outgoing emails from the PHP application. It allows developers to test email functionality without sending real messages to external mail servers. All captured emails are displayed in a convenient web interface.

In this project, Mailhog is deployed as a separate Docker container and is accessible at port 8025. Mailhog does not require any database or additional configuration. It simply listens for SMTP messages on port 1025 and displays them in its web UI.

Installing Composer

Composer is the standard dependency manager for PHP and is required to install PHPMailer and other third-party libraries. In this project, Composer was installed directly on the virtual machine running Ubuntu Server 22.04 LTS.

Navigate to the project directory:

```
cd ~/repif/www
```

Download the installer:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
```

Retrieve the latest installer hash:

```
HASH=$(curl -s https://composer.github.io/installer.sig)
```

Verify the installer:

```
php -r "if (hash_file('SHA384', 'composer-setup.php') === '$HASH') { echo 'Installer verified'; } else { echo 'Installer corrupt'; unlink('composer-setup.php'); } echo PHP_EOL;"
```

Run the installer:

```
sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer
```

Remove the installer:

```
php -r "unlink('composer-setup.php');"
```

Check the version:

```
composer --version
```

Install PHPMailer Using Composer

Navigate to the project's web root:

```
cd ~/repif/www
```

Install PHPMailer:

```
composer require phpmailer/phpmailer
```

This creates the directory:

```
~/repif/www/vendor/
```

which contains all PHPMailer classes and autoload files.

Check that Composer created the autoloader:

```
ls ~/repif/www/vendor/autoload.php
```

PHPMailer Test Script

A test script was created to verify SMTP email sending through Mailhog:

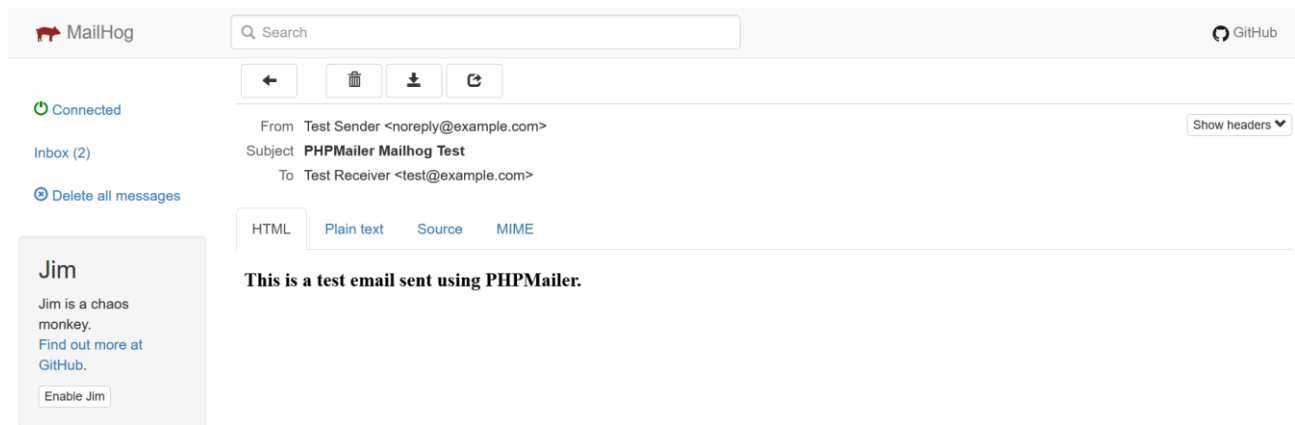
```
~/repif/www/public/mailtest.php
```

Here is the full script:

```
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;
require __DIR__ . '/../vendor/autoload.php';
$mail = new PHPMailer(true);
try {
    $mail->isSMTP();
    $mail->Host = 'mailhog';
    $mail->Port = 1025;
    $mail->SMTPAuth = false;
    $mail->setFrom('noreply@example.com', 'Test Sender');
    $mail->addAddress('test@example.com', 'Test Receiver');
    $mail->isHTML(true);
    $mail->Subject = 'PHPMailer Mailhog Test';
    $mail->Body = '<h3>This is a test email sent using PHPMailer.</h3>';
    $mail->AltBody = 'This is a test email sent using PHPMailer.';
    $mail->send();
    echo "<h2>Email sent successfully via PHPMailer!</h2>";
} catch (Exception $e) {
    echo "<h2>Failed to send email:</h2>";
    echo $mail->ErrorInfo;
}
?>
```

Email sent successfully via PHPMailer!

Picture 11.5. Email is sent



Picture 11.6. Email is recieved

Resources

Docker	https://docs.docker.com
Docker Compose	https://docs.docker.com/compose
Apache HTTP Server	https://httpd.apache.org/docs
PHP	https://www.php.net/docs.php
MariaDB	https://mariadb.com/docs/
phpMyAdmin	https://docs.phpmyadmin.net
Mailhog	https://github.com/mailhog/MailHog
Composer	https://getcomposer.org/doc
PHPMailer	https://github.com/PHPMailer/PHPMailer

L5 DATA TRANSFER

Introduction

The goal is to implement a reliable and maintainable mechanism for transferring measurement data from customer stations to the central database. PHP-based data transfer endpoint that receives measurements via HTTP POST, validates them, and stores them in the existing database schema.

The data transfer script acts as the bridge between the physical measurement stations and the backend infrastructure. It ensures that each station can submit its serial number, timestamp, and all measured values in a consistent format. In addition to the machine-oriented API, a simple manual input form is provided to allow testing, debugging, and manual data entry without requiring a separate client application.

Task Description

The purpose of this task is to implement a reliable mechanism for transferring measurement data from customer stations to the central database. The system must accept data sent via HTTP POST, validate all received fields, and store the values in the appropriate database tables.

The required functionality includes:

- receiving the station's serial number, timestamp, and all measured values
- validating and normalizing the input
- inserting the measurement into the database
- assigning the measurement to the correct collections based on time slots

In addition to the automated API endpoint, a manual input form is provided. This form allows operators to submit measurement data through a browser interface for testing or manual corrections. It uses the same backend logic as the automated POST requests to ensure consistent processing.

Database Preparation

For development and testing, a separate test database was created on the server. This database contained only the essential tables required for receiving and storing measurement data.

Although the full system includes additional logic for assigning measurements to time-based slots and collections, this functionality was implemented only for internal testing purposes.

To support this, the test database included:

- a Station table for validating incoming serial numbers.
- a Measurement table for storing received values.
- additional Slot and Collection tables used to confirm that timestamps were processed correctly.

This setup ensured that the data transfer script could be fully tested without affecting production data and without requiring the complete system to be deployed. The full test database is provided below:

```
CREATE DATABASE IF NOT EXISTS testdb;

USE testdb;

GRANT SELECT, INSERT, UPDATE, DELETE ON testdb.* TO 'webuser'@'%';

FLUSH PRIVILEGES;

CREATE TABLE station (
    pk_stationID VARCHAR(50) PRIMARY KEY
);

CREATE TABLE collection (
    pk_collectionID INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    description VARCHAR(1000),
    fk_user VARCHAR(50) NOT NULL,
    createdAt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_collection_owner_name (fk_user, name)
);

CREATE TABLE slot (
    pk_slotID INT AUTO_INCREMENT PRIMARY KEY,
    fk_collection INT NOT NULL,
    fk_station VARCHAR(50) NOT NULL,
    startDateTime DATETIME NOT NULL,
    endDateTime DATETIME NOT NULL,
    FOREIGN KEY (fk_collection) REFERENCES collection(pk_collectionID)
        ON DELETE CASCADE,
    FOREIGN KEY (fk_station) REFERENCES station(pk_stationID)
        ON DELETE CASCADE
);

CREATE TABLE measurement (
```

```

pk_measurementID INT AUTO_INCREMENT PRIMARY KEY,
timestamp DATETIME NOT NULL,
temperature DECIMAL(5,2),
humidity DECIMAL(5,2),
airPressure DECIMAL(6,2),
lightIntensity DECIMAL(6,2),
airQuality DECIMAL(6,2),
fk_station VARCHAR(50) NOT NULL,
FOREIGN KEY (fk_station) REFERENCES station(pk_stationID)
    ON DELETE CASCADE
);
CREATE TABLE contains (
    pkfk_measurement INT NOT NULL,
    pkfk_collection INT NOT NULL,
    PRIMARY KEY (pkfk_measurement, pkfk_collection),
    FOREIGN KEY (pkfk_measurement) REFERENCES measurement(pk_measurementID)
        ON DELETE CASCADE,
    FOREIGN KEY (pkfk_collection) REFERENCES collection(pk_collectionID)
        ON DELETE CASCADE
);
INSERT INTO station (pk_stationID) VALUES ('TEST_STATION');
INSERT INTO collection (name, description, fk_user)
VALUES ('Test Collection', 'Test collection for slot logic', 'tester');
INSERT INTO slot (fk_collection, fk_station, startDateTime, endDateTime)
VALUES (1, 'TEST_STATION', '2026-01-29 11:00:00', '2026-01-29 11:02:00');

```

PHP Data Transfer Script

The data transfer script provides a single endpoint that receives measurement data via HTTP POST and stores it in the database. Its implementation focuses on correctness, simplicity, and compatibility with both automated station requests and manual submissions.

The script expects a fixed set of POST fields: the station serial number, a timestamp, and all measured values. Each field is validated to ensure that the request contains

complete and usable data. Numeric values are normalized by converting commas to dots, and the timestamp is processed to guarantee a consistent format.

Before inserting a measurement, the script verifies that the provided station exists. If the station is unknown or required fields are missing, the script returns an error. Valid measurements are inserted into the Measurement table using prepared statements to ensure security and reliability.

The script also checks whether the submitted timestamp falls within any predefined time slots and records the corresponding collection assignments.

The script distinguishes between automated and manual submissions. When called by a station, it returns a JSON response containing the measurement ID and additional metadata. When used through the manual input form, it redirects back to the form after processing, preventing JSON output from appearing in the browser.

The full script is provided below:

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    header('Content-Type: text/html; charset=utf-8');
    ?>
<!DOCTYPE html>
<html>
<head>
<title>Manual Sender</title>
<style>
    body {
        font-family: Arial, sans-serif;
        background: #f7f7f7;
        padding: 20px;
    }
    .form-box {
        background: white;
        padding: 20px;
        border-radius: 8px;
        max-width: 400px;
        margin: auto;
        box-shadow: 0 2px 6px rgba(0,0,0,0.1);
    }
}
```

```

}

.form-box label {
    font-weight: bold;
    margin-top: 10px;
    display: block;
}

.form-box input {
    width: 100%;
    padding: 8px;
    margin-top: 4px;
    border: 1px solid #ccc;
    border-radius: 4px;
}

.form-box button {
    margin-top: 15px;
    width: 100%;
    padding: 10px;
    background: #0078ff;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 15px;
}

.form-box button:hover {
    background: #005fcc;
}
</style>
</head>
<body>
<div class="form-box">
    <h2>Manual Data Sender</h2>

```

```

<form method="POST">
    <input type="hidden" name="from_form" value="1">
    <label>Station Serial:</label>
    <input type="text" name="station_serial" required>
    <label>Timestamp (YYYY-MM-DD HH:MM:SS):</label>
    <input type="text" id="timestamp" name="timestamp" required>
    <label>Temperature:</label>
    <input type="number" name="temperature" required>
    <label>Humidity:</label>
    <input type="number" name="humidity" required>
    <label>Pressure:</label>
    <input type="number" name="pressure" required>
    <label>Light:</label>
    <input type="number" name="light" required>
    <label>Gas:</label>
    <input type="number" name="gas" required>
    <button type="submit">Send</button>
</form>
</div>
</body>
</html>
<?php
    exit;
}

header('Content-Type: application/json');
$host = getenv('DB_HOST');
$db = getenv('DB_NAME');
$user = getenv('DB_USER');
$pass = getenv('DB_PASS');
$conn = mysqli_connect($host, $user, $pass, $db);
if (!$conn) {
    http_response_code(500);
    echo json_encode(["error" => mysqli_connect_error()]);
}

```

```

    exit;
}
$station = $_POST['station_serial'] ?? null;
$timestamp_raw = $_POST['timestamp'] ?? null;
$temperature = $_POST['temperature'] ?? null;
$humidity = $_POST['humidity'] ?? null;
$pressure = $_POST['pressure'] ?? null;
$light = $_POST['light'] ?? null;
$gas = $_POST['gas'] ?? null;
if (!$station || !$timestamp_raw) {
    http_response_code(400);
    echo json_encode(["error" => "Missing station_serial or timestamp"]);
    exit;
}
if (!str_contains($timestamp_raw, '.')) {
    $timestamp_raw .= '.000000';
}
$station_check = $conn->prepare("SELECT pk_stationID FROM station WHERE pk_stationID = ?");
$station_check->bind_param("s", $station);
$station_check->execute();
$station_exists = $station_check->get_result()->num_rows > 0;

if (!$station_exists) {
    http_response_code(400);
    echo json_encode([
        "error" => "Unknown station_serial"
    ]);
    exit;
}
$dt = DateTime::createFromFormat("Y-m-d H:i:s.u", $timestamp_raw);
if (!$dt) {
    http_response_code(400);

```

```

    echo json_encode(["error" => "Invalid timestamp format"]);
    exit;
}
$timestamp = $dt->format("Y-m-d H:i:s");
$stmt = $conn->prepare("
    INSERT INTO measurement (timestamp, temperature, humidity, airPressure, lightIntensity, airQuality,
fk_station)
    VALUES (?, ?, ?, ?, ?, ?, ?)
");
$stmt->bind_param("sddddds", $timestamp, $temperature, $humidity, $pressure, $light, $gas, $station);
$stmt->execute();
$measurement_id = $stmt->insert_id;
$slot_query = $conn->prepare("
    SELECT pk_slotID, fk_collection
    FROM slot
    WHERE fk_station = ?
        AND startDateTime <= ?
        AND endDateTime >= ?
");
$slot_query->bind_param("sss", $station, $timestamp, $timestamp);
$slot_query->execute();
$slots = $slot_query->get_result();

while ($slot = $slots->fetch_assoc()) {
    $collection_id = $slot['fk_collection'];
    $ins = $conn->prepare("
        INSERT INTO contains (pkfk_measurement, pkfk_collection)
        VALUES (?, ?)
    ");
    $ins->bind_param("ii", $measurement_id, $collection_id);
    $ins->execute();
}
if (isset($_POST['from_form'])) {

```

```
header("Location: recieve_data.php");
exit;
}
echo json_encode([
    "status" => "ok",
    "measurement_id" => $measurement_id,
]);
```

Test Submission Script

To verify that the data transfer endpoint behaved correctly before connecting real stations, a small auxiliary python script was used to simulate measurement submissions. This script allowed sending controlled POST requests directly to the data transfer URL, making it possible to test validation, timestamp handling, and database insertion without relying on hardware.

The script sends the same fields that a station would normally transmit: the station serial number, timestamp, and all measurement values. By adjusting these values manually, it was possible to test different scenarios, such as missing fields, invalid timestamps, or edge-case numeric values. This helped confirm that the data transfer script correctly processed valid input and rejected malformed requests.

```
import requests
from datetime import datetime
import time

SERVER_IP = "192.168.178.48"
API_PATH = "/recieve_data.php"
STATION_ID = "TEST_STATION"
SEND_INTERVAL = 2
SEND_COUNT = 1
TEST_DATA = {
    "temperature": 20.5,
    "humidity": 48.2,
    "pressure": 1001.3,
    "light": 12.7,
    "gas": 450.0
}

def send_test_data():
    url = f"http://{SERVER_IP}{API_PATH}"
```



```

payload = {
    "station_serial": STATION_ID,
    "timestamp": "2026-01-29 11:01:20.000000",
    "temperature": TEST_DATA["temperature"],
    "humidity": TEST_DATA["humidity"],
    "pressure": TEST_DATA["pressure"],
    "light": TEST_DATA["light"],
    "gas": TEST_DATA["gas"]
}

print(f"Sending: {payload}")

try:
    response = requests.post(url, data=payload, timeout=2)
    print("Server response:", response.text)
except Exception as e:
    print("Error sending data:", e)

if __name__ == "__main__":
    for i in range(SEND_COUNT):
        send_test_data()
        time.sleep(SEND_INTERVAL)

```

```

Sending: {'station_serial': 'TEST_STATION', 'timestamp': '2026-01-29 11:01:20.000000', 'temperature': 20.5, 'humidity': 48.2, 'pressure': 1001.3, 'light': 12.7, 'gas': 450.0}
Server response: {"status": "ok", "measurement_id": 8}

```

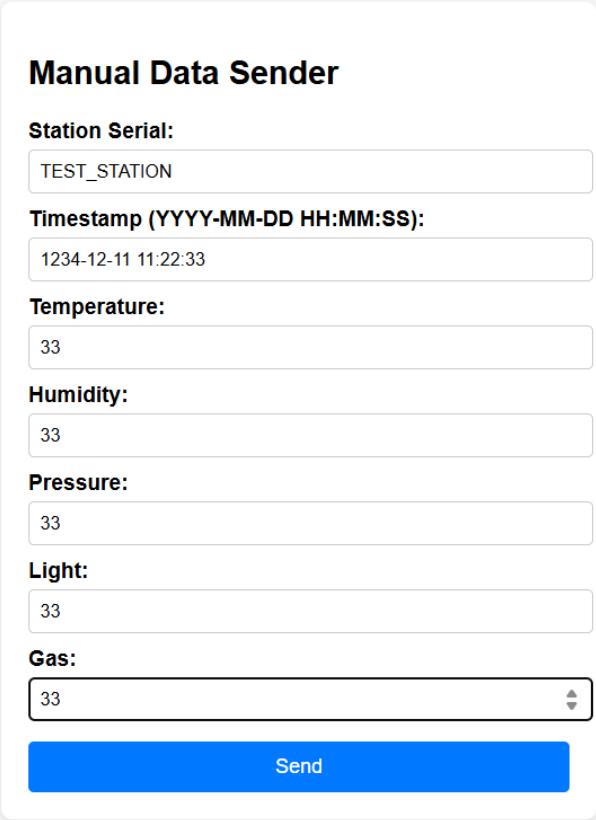
Picture 12.1. Sending test data

Manual Input Form

To complement the automated POST interface, a simple manual input form was created. Its purpose is to allow operators to submit measurement data directly through a web browser, making it possible to test the data transfer script without using external tools or hardware stations.

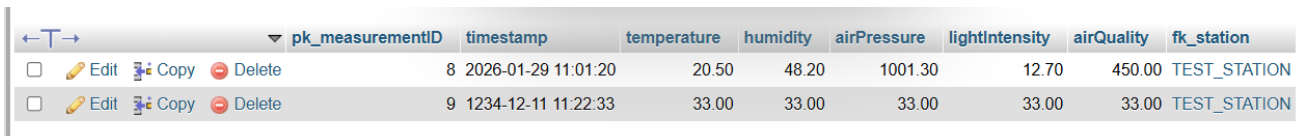
The form contains fields for the station serial number, timestamp, and all measurement values. Each field corresponds exactly to the POST parameters expected by the data transfer script, ensuring that manual submissions are processed in the same way as automated ones. Basic HTML validation is used to prevent empty submissions.

When the form is submitted, it sends a POST request to the same endpoint used by stations. After processing, the data transfer script redirects the user back to the form, keeping the interface clean and preventing JSON output from appearing in the browser.



The image shows a web browser window with the URL `http://192.168.178.48/recieve_data.php`. The page contains a form titled "Manual Data Sender". The form has several input fields: "Station Serial:" with the value "TEST_STATION", "Timestamp (YYYY-MM-DD HH:MM:SS):" with the value "1234-12-11 11:22:33", "Temperature:" with the value "33", "Humidity:" with the value "33", "Pressure:" with the value "33", "Light:" with the value "33", and "Gas:" with a dropdown menu showing "33". At the bottom of the form is a blue "Send" button.

Picture 12.2. Manual Input Form



The image shows a database table view with the following columns: `pk_measurementID`, `timestamp`, `temperature`, `humidity`, `airPressure`, `lightIntensity`, `airQuality`, and `fk_station`. There are two rows of data. The first row has values: 8, 2026-01-29 11:01:20, 20.50, 48.20, 1001.30, 12.70, 450.00, and TEST_STATION. The second row has values: 9, 1234-12-11 11:22:33, 33.00, 33.00, 33.00, 33.00, 33.00, and TEST_STATION. Each row has a checkbox, an edit icon, a copy icon, and a delete icon.

	<code>pk_measurementID</code>	<code>timestamp</code>	<code>temperature</code>	<code>humidity</code>	<code>airPressure</code>	<code>lightIntensity</code>	<code>airQuality</code>	<code>fk_station</code>
☐ Edit Copy Delete	8	2026-01-29 11:01:20	20.50	48.20	1001.30	12.70	450.00	TEST_STATION
☐ Edit Copy Delete	9	1234-12-11 11:22:33	33.00	33.00	33.00	33.00	33.00	TEST_STATION

Picture 12.3. Inserted measurements via script(8) and Manual form(9)

L6 BACKUP SERVER

Web-Database Server Configuration

The Web-Database server acts as the primary data source. It hosts the application and the `cheol904_db` MariaDB instance within a Docker environment. This architecture ensures that database services are isolated and easily manageable.

1.1. Docker & MariaDB Networking

To allow the Backup Server to reach the database, the MariaDB port is exposed in docker-compose.yml. Mapping the internal port to the host machine is essential for establishing a secure remote connection. services: db:

```
image: mariadb:11
```

```
container_name: db ports:
```

```
- "3306:3306" environment:
```

```
MYSQL_DATABASE: cheol904_db
```

```
# ... other env vars ...
```

1.2. Database User Privileges

A dedicated backup user is created with restricted access (Principle of Least Privilege). By limiting permissions to specific tables and actions, we significantly reduce the potential impact of credential compromise.

-- Execute inside the MariaDB container

```
CREATE USER 'backup_user'@'BACKUP_SERVER_IP' IDENTIFIED BY  
'secure_password_here';
```

```
GRANT SELECT, LOCK TABLES, SHOW VIEW ON cheol904_db.* TO  
'backup_user'@'BACKUP_SERVER_IP';
```

```
FLUSH PRIVILEGES;
```

1.3. Sudoers Configuration for Full OS Backup

To back up "all operating system files" without manual intervention, the backup user is granted passwordless execution rights for rsync. This configuration is vital for running automated synchronization tasks that require elevated system permissions. Command: sudo visudo

Add the following line at the end:

```
user ALL=(ALL) NOPASSWD: /usr/bin/rsync
```

Backup Server Configuration

2.1. Service Installation

The following services are required to manage synchronization and storage. These tools provide the necessary environment for secure file transfer and database management. sudo apt update sudo apt install mysql-client vsftpd rsync ssh -y

2.2. Secure SSH Key Authentication

Automated scripts require passwordless SSH access to the Web Server. Utilizing SSH keys instead of passwords enhances security and enables seamless execution of background backup processes. Generate 4096-bit RSA key

```
ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
```

Copy the public key to the Web Server ssh-copy-id

```
-i ~/.ssh/id_rsa.pub user@web-server-ip
```

2.3. FTP Isolation (Chroot Jail)

The ftpbackup user is restricted to the backup directory for security. Implementing a chroot jail ensures that the user cannot navigate outside their designated folder, protecting sensitive system files. Config Parameter | Value local_enable | YES write_enable | YES chroot_local_user | YES allow_writeable_chroot | YES

Automation & Scripting

3.1. Database & Files Backup (backup.sh)

This primary script automates the daily backup of critical application data. It performs a database dump and synchronizes user-uploaded files while maintaining a log of all operations.

3.2. Full OS Backup (os_backup.sh)

The full system backup ensures that the entire operating system environment can be restored. To optimize the process, temporary and system-generated directories are excluded from the synchronization.

3.3. Crontab Schedule

The backup schedule is defined using cron jobs to ensure regular execution. These time intervals are chosen to perform backups during periods of low server activity.

Daily Data Backup at 08:00

```
00 08 * * * /home/user/backup.sh >> /var/log/cron_errors.log 2>&1
```

Weekly Full OS Backup on Sunday at 03:00

```
00 03 * * 0 /home/user/os_backup.sh >> /var/log/cron_errors.log 2>&1
```

Disaster Recovery

Effective recovery procedures are essential for maintaining business continuity. In case of data loss or system failure, use the following methods to restore your environment.

Database Restore: `mysql --login-path=db_login cheol904_db < db_backup.sql`

System Restore: In case of a full VM crash, revert to the latest VirtualBox/VMware Snapshot. Individual system files can be recovered from the FULL_OS directory on the Backup Server via rsync.

